

Uniwersytet Mikołaja Kopernika w Toruniu  
Wydział Matematyki i Informatyki

Norbert Jaskulski

nr albumu: 313097

kierunek: informatyka

Praca magisterska

# **Platforma do analizy i generowania struktur formalnych z wykorzystaniem Sztucznej Inteligencji**

Opiekun pracy dyplomowej  
dr Anna Kwiatkowska, prof.  
UMK

Toruń 2026

# Spis treści

|                                                                                                             |           |
|-------------------------------------------------------------------------------------------------------------|-----------|
| <b>Rozdział 1. Wprowadzenie</b>                                                                             | <b>3</b>  |
| 1.1. Kontekst i motywacja . . . . .                                                                         | 3         |
| 1.2. Cel i zakres pracy . . . . .                                                                           | 3         |
| 1.3. Hipoteza badawcza . . . . .                                                                            | 4         |
| 1.4. Zakres pracy i ograniczenia . . . . .                                                                  | 4         |
| 1.5. Struktura pracy . . . . .                                                                              | 4         |
| <b>Rozdział 2. Podstawy teoretyczne języków formalnych oraz metod ich analizy i generowania</b>             | <b>6</b>  |
| 2.1. Wprowadzenie . . . . .                                                                                 | 6         |
| 2.2. Zastosowanie języków bezkontekstowych w analizie składniowej . . . . .                                 | 6         |
| 2.3. Podstawowe pojęcia języków formalnych . . . . .                                                        | 6         |
| 2.4. Operacje na językach formalnych . . . . .                                                              | 7         |
| 2.5. Gramatyki formalne . . . . .                                                                           | 7         |
| 2.6. Wyrażenia regularne . . . . .                                                                          | 8         |
| 2.7. Automaty skończone . . . . .                                                                           | 8         |
| 2.8. Języki bezkontekstowe i gramatyki bezkontekstowe . . . . .                                             | 9         |
| 2.9. Automaty ze stosem . . . . .                                                                           | 10        |
| 2.10. Algorytm $L^*$ Angluina i uczenie języków formalnych . . . . .                                        | 10        |
| 2.11. Algorytm Cocke'a-Younger'a-Kasamiego (CYK) . . . . .                                                  | 12        |
| 2.12. Analiza i generowanie języków formalnych . . . . .                                                    | 14        |
| <b>Rozdział 3. Metody analizy i generowania struktur formalnych z wykorzystaniem sztucznej inteligencji</b> | <b>16</b> |
| 3.1. Wprowadzenie . . . . .                                                                                 | 16        |
| 3.2. Zastosowanie sztucznej inteligencji w językach formalnych . . . . .                                    | 16        |
| 3.3. Modele językowe i generatywna sztuczna inteligencja . . . . .                                          | 17        |
| 3.4. Automatyczne uczenie automatów i gramatyk . . . . .                                                    | 18        |
| 3.5. Tworzenie automatów ze stosem z wykorzystaniem gramatyk bezkontekstowych . . . . .                     | 19        |
| 3.6. Założenia projektowe platformy badawczej . . . . .                                                     | 20        |
| 3.7. Algorytmy grafowe w analizie struktur formalnych . . . . .                                             | 21        |
| 3.8. Algorytmy grafowe w analizie struktur formalnych . . . . .                                             | 23        |
| 3.8.1 Przeszukiwanie wszerek (Breadth-First Search) . . . . .                                               | 24        |
| 3.8.2 Przeszukiwanie wszerek w odwróconym grafie . . . . .                                                  | 25        |
| 3.8.3 Przeszukiwanie w głąb (Depth-First Search) . . . . .                                                  | 25        |
| <b>Rozdział 4. Projekt i działanie aplikacji</b>                                                            | <b>26</b> |
| 4.1. Architektura systemu . . . . .                                                                         | 26        |
| 4.2. Model danych . . . . .                                                                                 | 27        |

|                                                      |                                                                          |           |
|------------------------------------------------------|--------------------------------------------------------------------------|-----------|
| 4.3.                                                 | Moduł generowania struktur z wykorzystaniem sztucznej inteligencji . . . | 28        |
| 4.4.                                                 | Implementacja algorytmu L* . . . . .                                     | 32        |
| 4.5.                                                 | Moduł analizy automatów skończonych . . . . .                            | 37        |
| 4.6.                                                 | Moduł analizy automatów ze stosem . . . . .                              | 40        |
| 4.7.                                                 | Obsługa gramatyk bezkontekstowych . . . . .                              | 42        |
| 4.8.                                                 | Implementacja algorytmu CYK . . . . .                                    | 43        |
| 4.9.                                                 | Graficzny edytor struktur formalnych . . . . .                           | 46        |
| <b>Rozdział 5. Analiza rozwiązania</b>               |                                                                          | <b>47</b> |
| 5.1.                                                 | Metodologia ewaluacji . . . . .                                          | 47        |
| 5.2.                                                 | Iteracyjne poprawki algorytmu L* . . . . .                               | 48        |
| 5.3.                                                 | Wyniki końcowe dla modelu bazowego . . . . .                             | 49        |
| 5.4.                                                 | Wariancja między przebiegami . . . . .                                   | 50        |
| 5.5.                                                 | Eksperyment: doszkalanie modelu metodą LoRA . . . . .                    | 51        |
| 5.5.1                                                | Generowanie danych treningowych . . . . .                                | 51        |
| 5.5.2                                                | Proces treningu i napotkane trudności . . . . .                          | 52        |
| 5.5.3                                                | Wyniki i wnioski . . . . .                                               | 52        |
| 5.6.                                                 | Porównanie alternatywnych modeli bazowych . . . . .                      | 53        |
| 5.7.                                                 | Podsumowanie wniosków z analizy . . . . .                                | 54        |
| <b>Rozdział 6. Podsumowanie i wnioski</b>            |                                                                          | <b>55</b> |
| 6.1.                                                 | Ograniczenia i kierunki dalszego rozwoju . . . . .                       | 58        |
| <b>Rozdział 7. Instrukcja uruchomienia aplikacji</b> |                                                                          | <b>60</b> |
| 7.1.                                                 | Wymagania środowiskowe . . . . .                                         | 60        |
| 7.2.                                                 | Przygotowanie projektu . . . . .                                         | 60        |
| 7.3.                                                 | Instalacja i konfiguracja Ollama . . . . .                               | 61        |
| 7.4.                                                 | Konfiguracja modelu językowego . . . . .                                 | 61        |
| 7.5.                                                 | Uruchomienie aplikacji . . . . .                                         | 62        |
| 7.6.                                                 | Weryfikacja działania . . . . .                                          | 62        |
| 7.7.                                                 | Podstawowa procedura uruchomienia . . . . .                              | 63        |
| <b>Bibliografia</b>                                  |                                                                          | <b>64</b> |

# Rozdział 1. Wprowadzenie

## 1.1. Kontekst i motywacja

Rozwój technologii sztucznej inteligencji w ostatniej dekadzie diametralnie zmienił sposoby, w jakie przetwarzamy i interpretujemy dane językowe. Szczególnie przełomowa okazała się architektura transformera, wprowadzona w 2017 roku, która zrewolucjonizowała obszar przetwarzania języka naturalnego (NLP). Na jej podstawie powstały tzw. duże modele językowe (LLM, *Large Language Models*), takie jak GPT, Gemini czy Copilot, osiągające wysoki poziom kompetencji w rozumieniu i generowaniu tekstu w języku naturalnym.

Mimo znakomitych wyników w zadaniach związanych z językiem naturalnym, zdolność LLM do operowania na strukturach języków formalnych pozostaje stosunkowo słabo zbadaana. Języki formalne, stanowiące fundament m.in. kompilatorów, automatów skończonych, gramatyk formalnych czy opisu protokołów, wymagają ścisłych reguł składniowych i semantycznych, których przestrzeganie jest kluczowe dla poprawności tworzonych struktur. Integracja LLM z takimi domenami może przynieść korzyści w automatycznym projektowaniu i analizie formalnych modeli systemów, a także w dydaktyce informatyki teoretycznej i języków formalnych.

Pojawia się jednak wyzwanie: LLM są trenowane głównie na danych tekstowych w języku naturalnym, co nie gwarantuje ich zdolności do ścisłego przestrzegania reguł syntaktycznych języków formalnych. Badanie w jaki sposób można przystosować modele tego typu do interpretacji i generowania struktur formalnych, ma zatem istotne znaczenie zarówno teoretyczne, jak i praktyczne.

## 1.2. Cel i zakres pracy

Głównym celem niniejszej pracy jest opracowanie platformy do analizy i generowania struktur języków formalnych z wykorzystaniem sztucznej inteligencji, a w szczególności zbadanie zdolności dużych modeli językowych do transformacji tekstu w języku naturalnym na reprezentacje języków formalnych.

Praca ma charakter badawczo-rozwojowy i obejmuje dwa główne obszary:

- **Badawczy** - eksperymentalne sprawdzenie możliwości LLM w rozwiązywaniu zadań związanych z językami formalnymi, takich jak generowanie automatów skończonych, gramatyk bezkontekstowych czy drzew decyzyjnych na podstawie opisów w języku naturalnym.
- **Rozwojowy** - implementacja platformy programistycznej, która łączy możliwości LLM z klasycznymi metodami analizy struktur formalnych (np. algorytm  $L^*$ , metoda DCSA), umożliwiając ich wizualizację, weryfikację i eksperymentalną ocenę.

Dodatkowym celem jest ocena skuteczności różnych podejść do adaptacji LLM od *prompt*

*engineeringu* po lekkie metody strojenia (np. LoRA) w kontekście poprawności i spójności generowanych struktur formalnych.

### 1.3. Hipoteza badawcza

Hipoteza przyjęta w pracy zakłada, że:

Odpowiednio przystosowane duże modele językowe, w szczególności transformatory, mogą efektywnie interpretować opisy w języku naturalnym i generować struktury języków formalnych porównywalne pod względem poprawności z konstrukcjami uzyskiwanymi metodami klasycznymi.

Sprawdzenie tej hipotezy wymaga zaprojektowania eksperymentów porównawczych, które pozwolą ocenić:

- poprawność syntaktyczną i semantyczną generowanych struktur,
- wierność odwzorowania opisu w języku naturalnym,
- efektywność i stabilność transformacji przy różnych typach zadań i poziomach skomplikowania struktur.

### 1.4. Zakres pracy i ograniczenia

Zakres badań obejmuje wybrane klasy języków formalnych, przede wszystkim języki regularne (reprezentowane automatami skończonymi) oraz bezkontekstowe (CFG, *Context-Free Grammars*). Wynika to z ich fundamentalnego znaczenia w informatyce teoretycznej i przetwarzaniu języków.

Praca nie koncentruje się na trenowaniu modeli od podstaw, lecz na eksperymentach z istniejącymi LLM oraz na badaniu ich adaptacji do zadań strukturalnych. Ograniczenia wynikają również z dostępnych zasobów obliczeniowych, dlatego stosowane będą głównie modele typu open-source (np. LLaMA 2), które można dostosować do specyficznych zadań.

### 1.5. Struktura pracy

Praca składa się z siedmiu rozdziałów.

- W rozdziale 2 przedstawiono teoretyczne podstawy działania dużych modeli językowych i ich roli w przetwarzaniu języka naturalnego.
- Rozdział 3 opisuje proces transformacji języka naturalnego do języków formalnych, uwzględniając istniejące metody i potencjalne problemy.
- Rozdział 4 omawia projekt i implementację platformy badawczej, czyli narzędzia do analizy i generowania struktur formalnych z wykorzystaniem AI.

- W rozdziale 5 zaprezentowano wyniki eksperymentów przeprowadzonych z użyciem platformy, w tym ocenę skuteczności LLM w generowaniu struktur formalnych.
- Rozdział 6 zawiera podsumowanie i wnioski, w tym ocenę przyjętej hipotezy badawczej oraz wskazanie dalszych kierunków rozwoju.
- Rozdział 7 zawiera instrukcję uruchomienia aplikacji, obejmującą przygotowanie środowiska .NET, instalację Ollama oraz modelu `phi4-mini`, a także uruchomienie i podstawową weryfikację działania systemu.

## Rozdział 2. Podstawy teoretyczne języków formalnych oraz metod ich analizy i generowania

### 2.1. Wprowadzenie

Języki formalne stanowią jeden z fundamentalnych obszarów informatyki teoretycznej, będąc podstawą opisu procesów obliczeniowych oraz struktur symbolicznych. Ich znaczenie wynika z możliwości precyzyjnego definiowania zbiorów wyrażeń oraz jednoznacznej interpretacji reguł ich budowy. W przeciwieństwie do języków naturalnych, które cechują się wieloznacznością i zależnością od kontekstu, języki formalne opierają się na ścisłych zasadach matematycznych.

W niniejszym rozdziale przedstawione zostaną podstawowe definicje oraz modele matematyczne, które stanowią fundament dla projektowanej platformy służącej do analizy i generowania struktur języków formalnych z wykorzystaniem sztucznej inteligencji.

### 2.2. Zastosowanie języków bezkontekstowych w analizie składniowej

Języki bezkontekstowe są wykorzystywane do opisu struktur składniowych, w tym struktur zagnieżdżonych występujących w językach programowania [1, 2, 5]. Gramatyki bezkontekstowe pozwalają opisywać zależności, których nie można w ogólności reprezentować za pomocą samych języków regularnych.

W praktyce stosuje się różne algorytmy parsowania, takie jak parsery LL oraz LR, które umożliwiają analizę składniową na podstawie gramatyk bezkontekstowych [5]. Proces ten służy między innymi do sprawdzania poprawności składniowej programów oraz budowania ich reprezentacji wewnętrznej.

W kontekście projektowanej platformy języki bezkontekstowe oraz automaty ze stosom umożliwiają analizę struktur wymagających pamięci stosowej, rozszerzając zakres analizowanych modeli względem automatów skończonych [1, 2].

### 2.3. Podstawowe pojęcia języków formalnych

Podstawą każdego języka formalnego jest alfabet, definiowany jako skończony zbiór symboli:

$$\Sigma = \{a_1, a_2, \dots, a_n\}, \quad \Sigma \neq \emptyset \quad (1)$$

Na bazie alfabetu definiuje się słowa, czyli skończone ciągi symboli należących do alfabetu. Długość słowa  $w$  oznaczamy jako  $|w|$ . Szczególnym przypadkiem jest słowo puste:

$$\varepsilon, \quad |\varepsilon| = 0 \quad (2)$$

Zbiór wszystkich możliwych słów nad alfabetem  $\Sigma$  definiujemy jako:

$$\Sigma^* = \bigcup_{i=0}^{\infty} \Sigma^i \quad (3)$$

gdzie:

$$\Sigma^0 = \{\varepsilon\}, \quad \Sigma^1 = \Sigma, \quad \Sigma^2 = \Sigma \cdot \Sigma \quad (4)$$

Język formalny definiuje się jako dowolny podzbiór zbioru  $\Sigma^*$ :

$$L \subseteq \Sigma^* \quad (5)$$

## 2.4. Operacje na językach formalnych

Języki formalne można traktować jako zbiory, dlatego można na nich wykonywać różne operacje. Suma języków  $L_1$  i  $L_2$  definiowana jest jako:

$$L_1 \cup L_2 = \{w \mid w \in L_1 \vee w \in L_2\} \quad (6)$$

Konkatenacja języków ma postać:

$$L_1 \cdot L_2 = \{xy \mid x \in L_1, y \in L_2\} \quad (7)$$

Potęgowanie języka definiujemy jako:

$$L^n = \underbrace{L \cdot L \cdot \dots \cdot L}_{n \text{ razy}}, \quad L^0 = \{\varepsilon\} \quad (8)$$

Na tej podstawie definiujemy domknięcie Kleene'ego:

$$L^* = \bigcup_{n=0}^{\infty} L^n \quad (9)$$

oraz domknięcie dodatnie:

$$L^+ = \bigcup_{n=1}^{\infty} L^n \quad (10)$$

## 2.5. Gramatyki formalne

Gramatyka formalna jest narzędziem generowania języków i definiowana jest jako:

$$G = (V, \Sigma, P, S) \quad (11)$$

gdzie  $V$  oznacza zbiór symboli nieterminalnych,  $\Sigma$  zbiór symboli terminalnych,  $P$  zbiór produkcji, a  $S \in V$  symbol startowy.

Produkcje mają postać:

$$\alpha \rightarrow \beta \quad (12)$$

gdzie  $\alpha \in (V \cup \Sigma)^+$  oraz  $\beta \in (V \cup \Sigma)^*$ .

Proces generowania słowa opisujemy jako:

$$S \Rightarrow^* w, \quad w \in \Sigma^* \quad (13)$$

gdzie  $\Rightarrow^*$  oznacza domknięcie przechodnie relacji wyprowadzenia.

## 2.6. Wyrażenia regularne

Wyrażenia regularne stanowią formalny sposób opisu języków regularnych. Definiuje się je rekurencyjnie:

- $\emptyset$  jest wyrażeniem regularnym,
- $\varepsilon$  jest wyrażeniem regularnym,
- dla każdego  $a \in \Sigma$ , symbol  $a$  jest wyrażeniem regularnym.

Jeżeli  $r$  i  $s$  są wyrażeniami regularnymi, to:

$$r + s, \quad rs, \quad r^* \quad (14)$$

są również wyrażeniami regularnymi.

Język opisany przez wyrażenie regularne  $r$  oznaczamy jako  $L(r)$ .

## 2.7. Automaty skończone

Automat skończony definiuje się jako:

$$M = (Q, \Sigma, \delta, q_0, F) \quad (15)$$

gdzie:

- $Q$  jest zbiorem stanów,
- $\Sigma$  alfabetem,
- $\delta : Q \times \Sigma \rightarrow Q$  funkcją przejścia,
- $q_0 \in Q$  stanem początkowym,
- $F \subseteq Q$  zbiorem stanów akceptujących.

Dla automatów niedeterministycznych funkcja przejścia ma postać:

$$\delta : Q \times \Sigma \rightarrow 2^Q \quad (16)$$

Słowo  $w$  jest akceptowane przez automat, jeśli:

$$\hat{\delta}(q_0, w) \in F \quad (17)$$

## 2.8. Języki bezkontekstowe i gramatyki bezkontekstowe

Języki bezkontekstowe stanowią klasę języków formalnych wykorzystywaną między innymi do opisu składni języków programowania. Język bezkontekstowy jest definiowany jako język generowany przez gramatykę bezkontekstową [1, 2, 3].

Gramatyka bezkontekstowa (CFG – *Context-Free Grammar*) jest szczególnym przypadkiem gramatyki formalnej, w której wszystkie produkcje mają postać:

$$A \rightarrow \gamma \quad (18)$$

gdzie  $A \in V$  jest pojedynczym symbolem nieterminalnym, a  $\gamma \in (V \cup \Sigma)^*$  jest dowolnym ciągiem symboli terminalnych i nieterminalnych.

Język generowany przez gramatykę bezkontekstową definiuje się jako:

$$L(G) = \{w \in \Sigma^* \mid S \Rightarrow^* w\} \quad (19)$$

Gramatyki bezkontekstowe umożliwiają opis struktur zagnieżdżonych, takich jak poprawnie zbalansowane nawiasy. Przykładowa gramatyka generująca taki język może być zapisana jako:

$$S \rightarrow (S)S \mid \varepsilon \quad (20)$$

Istotnym elementem związanym z gramatykami bezkontekstowymi jest możliwość reprezentacji procesu generowania w postaci drzewa składniowego, które obrazuje strukturę wyprowadzenia słowa. Węzły drzewa odpowiadają symbolom nieterminalnym, natomiast liście symbolom terminalnym.

W analizie języków bezkontekstowych wykorzystuje się również specjalne postacie gramatyk, które upraszczają ich przetwarzanie. Jedną z nich jest postać normalna Chomsky'ego (CNF – *Chomsky Normal Form*), w której każda produkcja ma jedną z postaci [4, 1]:

$$A \rightarrow BC \quad \text{lub} \quad A \rightarrow a \quad (21)$$

gdzie  $A, B, C \in V$ , a  $a \in \Sigma$ .

Inną istotną postacią jest postać normalna Greibacha, w której produkcje mają postać:

$$A \rightarrow a\alpha \quad (22)$$

gdzie  $a \in \Sigma$ , a  $\alpha \in V^*$ .

Normalne formy gramatyk są wykorzystywane między innymi w konstrukcji algorytmów parsujących oraz w analizie własności języków [1, 5].

## 2.9. Automaty ze stosem

Automaty ze stosem (PDA – *Pushdown Automata*) stanowią model obliczeniowy zdolny do rozpoznawania języków bezkontekstowych. Rozszerzają one możliwości automatów skończonych poprzez wprowadzenie dodatkowej pamięci w postaci stosu.

Formalnie automat ze stosem definiuje się jako siódmkę:

$$M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F) \quad (23)$$

gdzie:

- $Q$  jest zbiorem stanów,
- $\Sigma$  alfabetem wejściowym,
- $\Gamma$  alfabetem stosu,
- $\delta : Q \times (\Sigma \cup \{\varepsilon\}) \times \Gamma \rightarrow 2^{Q \times \Gamma^*}$  funkcją przejścia,
- $q_0 \in Q$  stanem początkowym,
- $Z_0 \in \Gamma$  symbolem początkowym stosu,
- $F \subseteq Q$  zbiorem stanów akceptujących.

Działanie automatu ze stosem polega na przetwarzaniu symboli wejściowych oraz manipulowaniu zawartością stosu poprzez operacje dodawania (push) i usuwania (pop) symboli. Automat ze stosem akceptuje słowo  $w$ , jeśli istnieje ciąg przejść prowadzący od konfiguracji początkowej do konfiguracji akceptującej. Formalnie można to zapisać jako:

$$(q_0, w, Z_0) \Rightarrow^* (q, \varepsilon, \gamma) \quad (24)$$

gdzie  $q \in F$ .

Alternatywnie dopuszcza się akceptację przez pusty stos:

$$(q_0, w, Z_0) \Rightarrow^* (q, \varepsilon, \varepsilon) \quad (25)$$

Automaty ze stosem są ściśle powiązane z gramatykami bezkontekstowymi. Istnieje fundamentalne twierdzenie, które mówi, że dla każdego języka bezkontekstowego istnieje równoważny automat ze stosem oraz odwrotnie.

## 2.10. Algorytm $L^*$ Angluin i uczenie języków formalnych

Jednym z zagadnień związanych z analizą języków formalnych jest ich automatyczne uczenie na podstawie obserwacji. Klasycznym podejściem do tego problemu jest algorytm  $L^*$  zaproponowany przez Danę Angluin w 1987 roku. Algorytm ten umożliwia identyfikację

języka regularnego poprzez konstruowanie deterministycznego automatu skończonego na podstawie interakcji z tzw. nauczycielem (*teacher*) [8].

Model uczenia wykorzystywany w algorytmie  $L^*$  opiera się na dwóch typach zapytań: zapytaniach o przynależność oraz zapytaniach równoważności. Zapytanie o przynależność polega na sprawdzeniu, czy dane słowo  $w$  należy do języka  $L$ , co można formalnie zapisać jako:

$$MQ(w) = \begin{cases} 1, & \text{gdy } w \in L \\ 0, & \text{gdy } w \notin L \end{cases} \quad (26)$$

Zapytanie równoważności polega natomiast na sprawdzeniu, czy zaproponowany przez uczący się model automat  $M$  jest równoważny językowi docelowemu  $L$ , czyli:

$$L(M) = L \quad (27)$$

W przypadku, gdy automat nie jest równoważny, nauczyciel zwraca kontrprzykład  $w$ , taki że:

$$w \in L(M) \oplus L \quad (28)$$

gdzie  $\oplus$  oznacza różnicę symetryczną języków.

Algorytm  $L^*$  opiera się na strukturze zwanej tabelą obserwacji (*observation table*), która jest definiowana jako trójka:

$$(S, E, T) \quad (29)$$

gdzie  $S \subseteq \Sigma^*$  jest zbiorem prefiksów,  $E \subseteq \Sigma^*$  zbiorem sufiksów, a funkcja  $T : (S \cup S\Sigma) \times E \rightarrow \{0, 1\}$  określa przynależność słów do języka:

$$T(s, e) = \begin{cases} 1, & \text{gdy } se \in L \\ 0, & \text{gdy } se \notin L \end{cases} \quad (30)$$

Tabela obserwacji musi spełniać dwa kluczowe warunki: domknięcia oraz spójności. Warunek domknięcia oznacza, że dla każdego  $t \in S\Sigma$  istnieje taki  $s \in S$ , że wiersze odpowiadające  $t$  i  $s$  są identyczne:

$$\forall t \in S\Sigma \ \exists s \in S \ \text{row}(t) = \text{row}(s) \quad (31)$$

Warunek spójności zapewnia, że jeśli dwa wiersze są identyczne, to ich rozszerzenia o dowolny symbol również pozostają zgodne:

$$\text{row}(s_1) = \text{row}(s_2) \Rightarrow \forall a \in \Sigma \ \text{row}(s_1a) = \text{row}(s_2a) \quad (32)$$

Po spełnieniu tych warunków możliwe jest skonstruowanie deterministycznego automatu skończonego:

$$M = (Q, \Sigma, \delta, q_0, F) \quad (33)$$

gdzie zbiór stanów  $Q$  odpowiada unikalnym wierszom tabeli obserwacji.

Algorytm działa iteracyjnie, rozszerzając zbiory  $S$  i  $E$  oraz aktualizując tabelę obserwacji aż do momentu uzyskania poprawnego automatu. W przypadku otrzymania kontrprzykładu tabela jest aktualizowana poprzez dodanie odpowiednich prefiksów.

Złożoność zapytań algorytmu  $L^*$  jest wielomianowa względem liczby stanów minimalnego automatu oraz parametrów związanych z długością kontrprzykładów [8].

Algorytm  $L^*$  może być wykorzystywany do automatycznej inferencji modeli systemów na podstawie zapytań o przynależność oraz kontrprzykładów. W kontekście projektowanej platformy algorytm ten wykorzystano do automatycznego wnioskowania automatów skończonych na podstawie odpowiedzi dostarczanych przez wyrocznie [8].

## 2.11. Algorytm Cocke’a-Younger’a-Kasamiego (CYK)

Algorytm Cocke’a-Younger’a-Kasamiego (CYK – *Cocke-Younger-Kasami*) jest klasycznym algorytmem rozpoznawania języków bezkontekstowych. Służy do sprawdzania, czy dane słowo należy do języka generowanego przez określoną gramatykę bezkontekstową [9, 10].

Klasyczny algorytm CYK działa dla gramatyk zapisanych w postaci normalnej Chomsky’ego (CNF – *Chomsky Normal Form*), w której produkcje mają postać [9, 10]:

$$A \rightarrow BC \quad (34)$$

lub:

$$A \rightarrow a \quad (35)$$

gdzie  $A, B, C \in V$ , natomiast  $a \in \Sigma$ .

Algorytm opiera się na programowaniu dynamicznym i wykorzystuje tablicę trójkątną, w której przechowywane są informacje o symbolach nieterminalnych mogących generować fragmenty analizowanego słowa.

Niech:

$$w = a_1 a_2 \dots a_n \quad (36)$$

oznacza analizowane słowo długości  $n$ . Tworzona jest tablica:

$$T[i, j] \quad (37)$$

gdzie komórka  $T[i, j]$  zawiera zbiór wszystkich symboli nieterminalnych generujących fragment słowa długości  $j$ , rozpoczynający się od pozycji  $i$ .

W pierwszym kroku algorytm analizuje pojedyncze symbole wejściowe. Dla każdej produkcji:

$$A \rightarrow a_i \quad (38)$$

symbol  $A$  zostaje umieszczony w komórce:

$$T[i, 1] \quad (39)$$

Następnie algorytm iteracyjnie analizuje coraz dłuższe fragmenty słowa. Dla każdej produkcji postaci:

$$A \rightarrow BC \quad (40)$$

sprawdzone jest, czy istnieje podział fragmentu słowa na dwie części takie, że:

$$B \in T[i, k] \quad (41)$$

oraz:

$$C \in T[i + k, j - k] \quad (42)$$

Jeżeli warunek jest spełniony, symbol  $A$  dodawany jest do komórki:

$$T[i, j] \quad (43)$$

Proces jest powtarzany aż do wyznaczenia wartości dla całego słowa.

Słowo należy do języka generowanego przez gramatykę wtedy i tylko wtedy, gdy symbol startowy  $S$  znajduje się w komórce:

$$T[1, n] \quad (44)$$

Formalnie:

$$w \in L(G) \iff S \in T[1, n] \quad (45)$$

Przykładowo dla gramatyki:

$$S \rightarrow AB \quad (46)$$

$$A \rightarrow a \quad (47)$$

$$B \rightarrow b \quad (48)$$

oraz słowa:

$$w = ab \quad (49)$$

algorytm w pierwszym kroku umieszcza:

$$A \in T[1, 1] \quad (50)$$

oraz:

$$B \in T[2, 1] \quad (51)$$

Następnie, ponieważ istnieje produkcja:

$$S \rightarrow AB \quad (52)$$

otrzymujemy:

$$S \in T[1, 2] \quad (53)$$

co oznacza, że słowo  $ab$  należy do języka.

Złożoność czasowa algorytmu CYK wynosi:

$$O(n^3 \cdot |P|) \quad (54)$$

gdzie  $n$  oznacza długość analizowanego słowa, natomiast  $|P|$  liczbę produkcji gramatyki. Złożoność pamięciowa wynosi:

$$O(n^2) \quad (55)$$

Algorytm CYK może być wykorzystywany do rozpoznawania języków bezkontekstowych oraz jako element procedur parsowania opartych na programowaniu dynamicznym [9, 10]. W kontekście projektowanej platformy algorytm CYK może zostać wykorzystany do automatycznej weryfikacji przynależności słów do języka generowanego przez gramatykę bezkontekstową oraz do wizualizacji procesu parsowania i budowy struktur składniowych.

## 2.12. Analiza i generowanie języków formalnych

Analiza języka formalnego może obejmować sprawdzenie, czy dane słowo należy do języka [1, 2]:

$$w \in L \quad (56)$$

W przypadku gramatyk:

$$S \Rightarrow^* w \quad (57)$$

Parsowanie polega na budowie drzewa składniowego reprezentującego proces wyprowadzenia [5].

Generowanie języka polega na wyznaczeniu zbioru [1, 3]:

$$L(G) = \{w \in \Sigma^* \mid S \Rightarrow^* w\} \quad (58)$$

Proces ten może być realizowany poprzez systematyczne stosowanie reguł produkcji.

W projektowanej platformie wykorzystanie modelu językowego do interpretacji opisu w języku naturalnym można ująć jako odwzorowanie:

$$f : NL \rightarrow FL \quad (59)$$

gdzie  $NL$  oznacza język naturalny, a  $FL$  język formalny.

## Rozdział 3. Metody analizy i generowania struktur formalnych z wykorzystaniem sztucznej inteligencji

### 3.1. Wprowadzenie

Metody sztucznej inteligencji i uczenia maszynowego mogą wspomagać analizę, modelowanie oraz generowanie danych tekstowych i struktur reprezentowanych w sposób formalny [15, 14]. W kontekście niniejszej pracy rozważane są przede wszystkim ich zastosowania do inferencji oraz generowania struktur formalnych.

Klasyczne metody analizy języków formalnych opierają się na modelach matematycznych, takich jak automaty skończone, automaty ze stosem czy gramatyki formalne [1, 2]. Metody uczenia maszynowego mogą natomiast wspomagać proces tworzenia modeli na podstawie danych lub interakcji z systemem uczącym [8, 15].

Inferencja języków formalnych polega na automatycznym uczeniu modelu opisującego dany język na podstawie przykładów lub interakcji z nauczycielem. Jednym z klasycznych algorytmów tego typu jest  $L^*$ , zaproponowany przez Danę Angluin, który umożliwia aktywne uczenie automatów skończonych reprezentujących języki regularne [8]. Klasyczna wersja algorytmu dotyczy więc języków regularnych i automatów skończonych.

W przypadku języków bezkontekstowych oraz struktur wymagających pamięci stosowej stosuje się modele takie jak automaty ze stosem [1, 2]. Rozszerzenie metod aktywnego uczenia na tego rodzaju struktury wymaga uwzględnienia dodatkowej pamięci oraz zależności zagnieżdżonych, dlatego różni się od klasycznego uczenia automatów skończonych. Celem niniejszego rozdziału jest przedstawienie metod analizy i generowania struktur formalnych z wykorzystaniem sztucznej inteligencji, ze szczególnym uwzględnieniem uczenia automatów i gramatyk formalnych. Omówione zostaną klasyczne podejścia inferencyjne oraz metody wykorzystujące modele generatywne.

### 3.2. Zastosowanie sztucznej inteligencji w językach formalnych

Metody sztucznej inteligencji i uczenia maszynowego mogą być wykorzystywane do wspomagania analizy tekstu, kodu oraz struktur reprezentowanych formalnie [15, 14]. W niniejszej pracy rozważane są przede wszystkim zastosowania tych metod do generowania i inferencji struktur formalnych.

Jednym z obszarów zastosowań metod uczenia jest inferencja gramatyk formalnych. Inferencja polega na tworzeniu modelu języka na podstawie dostarczonych przykładów lub interakcji z systemem uczącym. W przypadku aktywnego uczenia model może być konstruowany na podstawie odpowiedzi uzyskiwanych od nauczyciela [8].

Metody aktywnego uczenia automatów (*active automata learning*) zakładają interakcję systemu uczącego z nauczycielem. System może zadawać pytania dotyczące przynależności

słów do języka oraz wykorzystywać kontrprzykłady do aktualizacji modelu [8]. Algorytm  $L^*$  jest klasycznym przykładem takiego podejścia.

Modele generatywne oparte na architekturze transformera są wykorzystywane do generowania tekstu i innych sekwencji na podstawie wzorców poznanych podczas uczenia [12, 13]. W zadaniach wymagających ścisłego przestrzegania reguł formalnych wygenerowany wynik może jednak wymagać dodatkowej walidacji za pomocą metod symbolicznych. W niniejszej pracy przyjęto właśnie takie rozdzielenie generowania i formalnej weryfikacji. Sztuczna inteligencja znajduje zastosowanie także w automatycznej analizie kodu źródłowego, wykrywaniu błędów składniowych, generowaniu parserów oraz optymalizacji procesów kompilacji. W systemach tych języki formalne wykorzystywane są do reprezentacji składni programów, natomiast algorytmy uczenia maszynowego wspomagają analizę wzorców oraz automatyczne wykrywanie nieprawidłowości.

Rozszerzenie metod aktywnego uczenia na języki bezkontekstowe i automaty ze stosem wymaga uwzględnienia pamięci stosowej oraz struktur zagnieżdżonych. Modele te pozwalają reprezentować klasy języków szersze niż języki regularne [1, 2]. W niniejszej pracy problem ten jest rozpatrywany z wykorzystaniem pośredniego podejścia opartego na gramatyce bezkontekstowej.

W niniejszej pracy sztuczna inteligencja wykorzystywana jest jako element wspomagający analizę i generowanie struktur formalnych w ramach opracowanej platformy badawczej. Szczególną uwagę poświęcono możliwości rozszerzenia klasycznego algorytmu  $L^*$  na modele wykorzystujące mechanizm stosu, co pozwala na analizę bardziej złożonych języków formalnych niż języki regularne.

### 3.3. Modele językowe i generatywna sztuczna inteligencja

[12, 13, 15]

Rozwój modeli językowych doprowadził do powstania systemów zdolnych do analizy i generowania danych tekstowych. Wśród współczesnych modeli generatywnych istotną rolę odgrywa architektura transformera [12, 13, 15].

Modele językowe (*Language Models - LM*) uczą się zależności występujących w danych tekstowych i mogą być wykorzystywane między innymi do przewidywania kolejnych elementów sekwencji oraz generowania tekstu [13, 14]. W zależności od architektury mogą wykorzystywać między innymi sieci rekurencyjne, sieci LSTM lub transformery [12, 14]. Istotnym etapem rozwoju współczesnych modeli językowych było zaproponowanie architektury transformera opartej na mechanizmie uwagi (*attention mechanism*) [12]. Architektura ta stała się podstawą wielu późniejszych modeli językowych, w tym modeli generatywnych z rodziny GPT [13].

Modele generatywne mogą być wykorzystywane do tworzenia nowych sekwencji na podstawie wzorców poznanych podczas uczenia [13, 15]. W kontekście niniejszej pracy wykorzystuje się je do generowania opisów struktur formalnych, takich jak automaty i gramatyki. Wykorzystanie modeli generatywnych do zadań wymagających ścisłego przestrzegania re-

guł formalnych wiąże się z koniecznością dodatkowej weryfikacji wyników. W przeciwieństwie do formalnie zdefiniowanych automatów i gramatyk, generowanie odpowiedzi przez model językowy nie stanowi samo w sobie dowodu spełnienia wszystkich reguł określonej struktury. Z tego względu w niniejszej pracy wygenerowane struktury są poddawane dalszej analizie algorytmicznej.

W przypadku struktur zagnieżdżonych dodatkowym wymaganiem jest zachowanie zależności pomiędzy odległymi elementami sekwencji. Dlatego wynik wygenerowany przez model językowy może wymagać sprawdzenia względem formalnej definicji języka, co w niniejszej pracy realizowane jest za pomocą klasycznych algorytmów.

W niniejszej pracy modele sztucznej inteligencji wykorzystywane są jako element wspomagający analizę i generowanie struktur formalnych w opracowanej platformie badawczej. Szczególne znaczenie ma możliwość połączenia klasycznych metod inferencji automatów z nowoczesnymi technikami sztucznej inteligencji w celu rozszerzenia możliwości analizy języków bezkontekstowych oraz automatów ze stosem.

### 3.4. Automatyczne uczenie automatów i gramatyk

Automatyczne uczenie automatów i gramatyk polega na konstruowaniu modelu formalnego opisującego język na podstawie dostępnych danych lub interakcji z systemem nauczycielskim. W literaturze proces ten określany jest między innymi jako inferencja gramatyk (*grammar inference*) lub uczenie automatów (*automata learning*) [8, 15].

W podejściu automatycznym model jest konstruowany na podstawie danych lub odpowiedzi uzyskiwanych podczas interakcji z nauczycielem, co ogranicza konieczność ręcznego definiowania całej struktury modelu [8].

Wyróżnia się między innymi uczenie pasywne (*passive learning*) oraz uczenie aktywne (*active learning*). W podejściu pasywnym system uczący otrzymuje zbiór przykładów i na ich podstawie konstruuje model formalny, natomiast w podejściu aktywnym może pozyskiwać dodatkowe informacje poprzez interakcję z nauczycielem [8].

Uczenie aktywne opiera się natomiast na interakcji pomiędzy systemem uczącym a nauczycielem (*teacher*). Algorytm może zadawać pytania dotyczące przynależności słów do języka (*membership queries*) oraz proponować hipotezy opisujące aktualny model języka. W przypadku błędnej hipotezy nauczyciel zwraca kontrprzykład (*counterexample*), który wykorzystywany jest do dalszego udoskonalania modelu.

Algorytm  $L^*$  zaproponowany przez Danę Angluin umożliwia aktywne uczenie deterministycznych automatów skończonych reprezentujących języki regularne. Jego działanie opiera się na budowie tablicy obserwacji (*observation table*), która przechowuje informacje o przynależności odpowiednich słów do języka [8].

Aktywne uczenie pozwala konstruować model na podstawie obserwacji zachowania systemu, bez konieczności bezpośredniej znajomości jego wewnętrznej implementacji [8].

Pomimo dużej skuteczności klasyczny algorytm  $L^*$  posiada istotne ograniczenia wynikające z faktu, że operuje wyłącznie na językach regularnych reprezentowanych przez

automaty skończone. W praktyce wiele rzeczywistych struktur formalnych, takich jak składnia języków programowania czy struktury zagnieżdżone, wymaga zastosowania języków bezkontekstowych oraz automatów ze stosem. W takich przypadkach klasyczne modele automatów skończonych okazują się niewystarczające.

Rozszerzenie metod aktywnego uczenia na automaty ze stosem oraz gramatyki bezkontekstowe wymaga uwzględnienia pamięci stosowej i zależności zagnieżdżonych, których nie reprezentują automaty skończone [1, 2].

Połączenie metod symbolicznych z metodami uczenia maszynowego pozwala rozdzielić generowanie lub przewidywanie odpowiedzi od formalnej weryfikacji wyników. W niniejszej pracy zastosowano takie rozdzielenie: model językowy dostarcza odpowiedzi lub strukturę, natomiast poprawność jest następnie sprawdzana za pomocą algorytmów formalnych.

W niniejszej pracy problem automatycznego uczenia automatów i gramatyk formalnych stanowi jeden z głównych obszarów badawczych. Szczególną uwagę poświęcono możliwości rozszerzenia klasycznego algorytmu  $L^*$  na modele wykorzystujące mechanizm stosu, co pozwala na analizę bardziej złożonych języków formalnych reprezentowanych przez automaty ze stosem.

### 3.5. Tworzenie automatów ze stosem z wykorzystaniem gramatyk bezkontekstowych

Algorytm  $L^*$  zaproponowany przez Danę Angluin umożliwia konstruowanie deterministycznych automatów skończonych na podstawie odpowiedzi na pytania o przynależność słów do języka oraz kontrprzykładów dostarczanych przez nauczyciela [8]. Otrzymany model opisuje język regularny, dlatego metoda ta nie obejmuje bezpośrednio struktur wymagających pamięci stosowej.

W przypadku struktur wymagających reprezentacji zależności zagnieżdżonych stosuje się języki bezkontekstowe i odpowiadające im automaty ze stosem (*Pushdown Automata* – *PDA*) [1, 2]. Dodatkowa pamięć w postaci stosu pozwala reprezentować zależności, których nie można w ogólności odwzorować za pomocą automatów skończonych.

W prezentowanej platformie badawczej przyjęto odmienne podejście do konstruowania automatów ze stosem niż w przypadku klasycznego algorytmu  $L^*$ . W obu przypadkach celem jest uzyskanie modelu formalnego odpowiadającego opisowi języka, jednak droga prowadząca do jego utworzenia jest inna. Algorytm  $L^*$  buduje automat stopniowo, wykorzystując odpowiedzi na zapytania o przynależność słów do języka oraz analizując kontrprzykłady. Proces inferencji polega zatem na bezpośrednim wyznaczaniu struktury automatu.

W przypadku języków bezkontekstowych zastosowano podejście pośrednie. Zamiast bezpośredniej inferencji automatu ze stosem tworzona jest najpierw gramatyka bezkontekstowa opisująca analizowany język. Następnie poprawność wygenerowanej gramatyki jest weryfikowana z wykorzystaniem algorytmu CYK, który umożliwia sprawdzenie przynależności przykładowych słów do języka opisanego przez gramatykę. Dopiero po pozytywnej

weryfikacji gramatyka staje się podstawą do skonstruowania odpowiadającego jej automatu ze stosem.

Takie podejście różni się od działania algorytmu  $L^*$  przede wszystkim obiektem podlegającym uczeniu. W algorytmie  $L^*$  wynikiem procesu jest bezpośrednio automat skończony, którego struktura jest sukcesywnie udoskonalana na podstawie kolejnych informacji uzyskiwanych od nauczyciela. W proponowanym rozwiązaniu modelem tworzonym przez system jest gramatyka bezkontekstowa, natomiast automat ze stosem powstaje jako formalna reprezentacja tej gramatyki. Oznacza to, że proces budowy automatu nie opiera się na analizie kontrprzykładów ani rozszerzaniu tablicy obserwacji, lecz na wykorzystaniu znanych własności gramatyk bezkontekstowych oraz formalnych algorytmów ich analizy. Zastosowanie algorytmu CYK pełni w tym procesie rolę mechanizmu weryfikacyjnego. Pozwala on ocenić, czy wygenerowana gramatyka poprawnie opisuje analizowany język przed przekształceniem jej do postaci automatu ze stosem. Dzięki temu możliwe jest ograniczenie wpływu błędów powstałych podczas generowania gramatyki przez model sztucznej inteligencji oraz zwiększenie wiarygodności otrzymywanego modelu formalnego. Przyjęta metoda uzupełnia podejście reprezentowane przez algorytm  $L^*$ .  $L^*$  służy do uczenia automatów dla języków regularnych [8], natomiast w proponowanym rozwiązaniu dla języków bezkontekstowych wykorzystano gramatykę, algorytm CYK oraz konstrukcję automatu ze stosem. Oba podejścia prowadzą do utworzenia formalnego modelu języka, lecz wykorzystują odmienne modele i procedury.

### 3.6. Założenia projektowe platformy badawczej

Jednym z głównych celów niniejszej pracy jest opracowanie platformy badawczej umożliwiającej analizę oraz generowanie struktur formalnych z wykorzystaniem metod sztucznej inteligencji. Projektowana aplikacja ma stanowić środowisko eksperymentalne pozwalające na badanie procesu inferencji języków formalnych, analizę działania automatów oraz testowanie rozszerzonego algorytmu  $L^*$  przeznaczonego dla automatów ze stosem.

Podstawowym założeniem projektowym platformy jest modularność architektury systemu. Aplikacja została zaprojektowana w sposób umożliwiający niezależny rozwój poszczególnych komponentów odpowiedzialnych za analizę języków formalnych, obsługę automatów, generowanie struktur formalnych oraz integrację z modułami sztucznej inteligencji. Takie podejście pozwala na łatwe rozszerzanie systemu o nowe algorytmy oraz modele formalne bez konieczności modyfikacji całej architektury aplikacji.

Istotnym założeniem platformy jest obsługa różnych klas języków formalnych, w szczególności języków regularnych oraz języków bezkontekstowych. System powinien umożliwiać definiowanie automatów skończonych, automatów ze stosem, gramatyk formalnych oraz wyrażeń regularnych. Dzięki temu możliwe staje się przeprowadzanie eksperymentów porównawczych pomiędzy różnymi metodami reprezentacji języków formalnych.

W projektowanej platformie szczególną uwagę poświęcono implementacji mechanizmów aktywnego uczenia automatów. System ma umożliwiać zadawanie pytań o przynależność

słów do języka, analizę kontrprzykładów oraz automatyczne budowanie modeli formalnych na podstawie uzyskanych odpowiedzi. Kluczowym elementem platformy jest implementacja rozszerzonego algorytmu  $L^*$  umożliwiającego analizę struktur wykorzystujących mechanizm stosowy.

Kolejnym ważnym założeniem projektowym jest możliwość wizualizacji działania automatów oraz procesu inferencji języka. Platforma powinna umożliwiać graficzną prezentację stanów automatów, przejść pomiędzy stanami, operacji wykonywanych na stosie oraz zmian zachodzących w strukturach danych wykorzystywanych przez algorytm uczenia. Wizualizacja procesu analizy ma ułatwić interpretację wyników eksperymentów oraz analizę skuteczności proponowanego rozwiązania.

System został również zaprojektowany z myślą o integracji z nowoczesnymi modelami sztucznej inteligencji. Platforma powinna umożliwiać wykorzystanie modeli językowych do wspomagania generowania struktur formalnych, automatycznej analizy składniowej oraz identyfikacji wzorców występujących w danych wejściowych. Dzięki temu możliwe staje się połączenie klasycznych metod symbolicznych z technikami uczenia maszynowego. Istotnym wymaganiem projektowym jest również możliwość przeprowadzania eksperymentów badawczych oraz analizy wyników działania systemu. Platforma ma umożliwiać generowanie przykładowych języków formalnych, testowanie poprawności działania automatów oraz porównywanie skuteczności różnych metod inferencji. System powinien także pozwalać na zapisywanie wyników eksperymentów oraz ich dalszą analizę.

W ramach pracy założono również możliwość wykorzystania platformy jako narzędzia edukacyjnego wspomagającego naukę teorii języków formalnych oraz automatów. Interaktywna forma prezentacji struktur formalnych i procesu działania algorytmów może ułatwić zrozumienie zagadnień związanych z analizą składniową, inferencją języków oraz teorią automatów.

Opracowana platforma badawcza stanowi podstawę części implementacyjnej niniejszej pracy. Szczegółowy opis architektury systemu, zastosowanych technologii oraz działania poszczególnych modułów przedstawiono w kolejnym rozdziale.

### **3.7. Algorytmy grafowe w analizie struktur formalnych**

W analizie języków formalnych oraz automatów istotną rolę odgrywają struktury grafowe. Automaty skończone, automaty ze stosem oraz drzewa składniowe mogą być reprezentowane w postaci grafów skierowanych, w których wierzchołki odpowiadają stanom lub symbolom nieterminalnym, natomiast krawędzie reprezentują przejścia pomiędzy stanami lub reguły wyprowadzenia.

Reprezentacja grafowa umożliwia wykorzystanie klasycznych algorytmów przeszukiwania grafów do analizy struktur formalnych. W projektowanej platformie badawczej algorytmy grafowe wykorzystywane są między innymi do eksploracji przestrzeni stanów automatów, wyszukiwania ścieżek akceptujących oraz analizy zależności pomiędzy symbolami gramatyk formalnych.

Jednym z podstawowych algorytmów stosowanych w analizie struktur formalnych jest przeszukiwanie wszerz (*Breadth-First Search* – *BFS*). Algorytm BFS eksploruje graf poziomami, odwiedzając najpierw wszystkie wierzchołki znajdujące się w tej samej odległości od wierzchołka początkowego [11]. W kontekście automatów formalnych BFS może być wykorzystywany do wyszukiwania najkrótszej ścieżki prowadzącej do stanu akceptującego oraz do analizy osiągalności stanów.

Formalnie graf można zdefiniować jako:

$$G = (V, E) \quad (60)$$

gdzie:

$$V \quad (61)$$

oznacza zbiór wierzchołków, natomiast:

$$E \subseteq V \times V \quad (62)$$

oznacza zbiór krawędzi.

Dla automatu skończonego wierzchołki odpowiadają stanom automatu:

$$V = Q \quad (63)$$

natomiast krawędzie odpowiadają funkcji przejścia:

$$(q_i, q_j) \in E \iff \exists a \in \Sigma; \delta(q_i, a) = q_j \quad (64)$$

Drugim istotnym algorytmem jest przeszukiwanie w głąb (*Depth-First Search* – *DFS*), które polega na eksploracji możliwie najgłębszych ścieżek grafu przed powrotem do wcześniejszych wierzchołków. Algorytm DFS znajduje zastosowanie między innymi w analizie struktur rekurencyjnych, wykrywaniu cykli oraz budowie drzew składniowych.

W przypadku języków bezkontekstowych algorytmy DFS mogą być wykorzystywane do analizy wyprowadzeń gramatycznych oraz eksploracji drzew parsowania. Szczególnie istotne jest to w analizie struktur zagnieżdżonych, charakterystycznych dla języków programowania oraz struktur XML.

W projektowanej platformie badawczej algorytmy BFS i DFS zostały wykorzystane jako podstawowe mechanizmy eksploracji struktur formalnych reprezentowanych w postaci grafów stanów. Umożliwia to analizę osiągalności stanów, wyszukiwanie ścieżek akceptujących oraz wizualizację działania automatów.

Istotnym elementem współczesnych systemów analizy struktur formalnych jest również połączenie algorytmów grafowych z metodami sztucznej inteligencji. W podejściach tych algorytmy eksploracji grafów wspomagane są przez modele uczenia maszynowego odpowiedzialne za ocenę istotności stanów, przewidywanie najbardziej prawdopodobnych przejść lub ograniczanie przestrzeni przeszukiwania.

W systemach inferencji automatów możliwe jest traktowanie procesu uczenia jako problemu eksploracji grafu stanów. Wierzchołki reprezentują potencjalne konfiguracje automatu, natomiast krawędzie odpowiadają przejściom wynikającym z analizy kolejnych symboli wejściowych. Modele sztucznej inteligencji mogą wspomagać wybór najbardziej obiecujących ścieżek eksploracji, co pozwala na zmniejszenie złożoności procesu uczenia. Podejścia hybrydowe łączące algorytmy grafowe z metodami sztucznej inteligencji znajdują zastosowanie między innymi w:

- inferencji automatów i gramatyk formalnych,
- analizie składniowej języków programowania,
- automatycznej analizie kodu źródłowego,
- eksploracji przestrzeni stanów systemów reaktywnych,
- weryfikacji formalnej oprogramowania,
- analizie struktur hierarchicznych i drzewiastych.

W kontekście niniejszej pracy algorytmy grafowe stanowią podstawowy mechanizm analizy struktur formalnych w opracowanej platformie badawczej. Integracja metod eksploracji grafów z mechanizmami sztucznej inteligencji umożliwia efektywniejszą analizę języków formalnych oraz wspomaga proces inferencji automatów i gramatyk.

### 3.8. Algorytmy grafowe w analizie struktur formalnych

W analizie języków formalnych oraz automatów istotną rolę odgrywają struktury grafowe. Automaty skończone, automaty ze stosem oraz drzewa składniowe mogą być reprezentowane jako grafy skierowane, w których wierzchołki odpowiadają stanom lub symbolom nieterminalnym, natomiast krawędzie opisują relacje przejść pomiędzy stanami lub zależnościami wynikające z reguł gramatycznych.

Reprezentacja grafowa umożliwia wykorzystanie klasycznych algorytmów przeszukiwania grafów do analizy właściwości struktur formalnych. W opracowanej platformie algorytmy grafowe stanowią jeden z podstawowych mechanizmów analizy automatów skończonych oraz automatów ze stosem. Wykorzystywane są między innymi do określania osiągalności stanów, wykrywania stanów pułapek, analizy skończoności języka, generowania zbiorów testowych oraz symulacji działania automatów.

Formalnie graf można zdefiniować jako:

$$G = (V, E), \quad (65)$$

gdzie  $V$  oznacza zbiór wierzchołków, natomiast

$$E \subseteq V \times V \quad (66)$$

oznacza zbiór krawędzi.

Dla deterministycznego automatu skończonego zbiór wierzchołków odpowiada zbiorowi stanów

$$V = Q, \quad (67)$$

natomiast krawędzie odpowiadają funkcji przejścia

$$(q_i, q_j) \in E \iff \exists a \in \Sigma \delta(q_i, a) = q_j. \quad (68)$$

### 3.8.1 Przeszukiwanie wszere (Breadth-First Search)

Najczęściej wykorzystywanym algorytmem grafowym w platformie jest przeszukiwanie wszere (Breadth-First Search – BFS). Algorytm eksploruje graf poziomami, odwiedzając najpierw wszystkie wierzchołki znajdujące się w tej samej odległości od wierzchołka początkowego. Dzięki temu możliwe jest systematyczne przeszukiwanie całej przestrzeni stanów.

Pierwszym zastosowaniem algorytmu BFS jest analiza osiągalności stanów w deterministycznych automatach skończonych. Przeszukiwanie rozpoczyna się od stanu początkowego i odwiedza kolejne stany osiągalne poprzez funkcję przejścia. Wszystkie nieodwiedzone stany są klasyfikowane jako stany nieosiągalne, które nie mają wpływu na rozpoznawany język i mogą zostać wykryte podczas analizy automatu.

Drugim zastosowaniem jest generowanie przykładowych słów akceptowanych i odrzucanych przez automat. W tym przypadku elementami kolejki są pary składające się z aktualnie wygenerowanego słowa oraz odpowiadającego mu stanu automatu. W każdej iteracji do słowa dołączany jest kolejny symbol alfabetu, a następnie wykonywane jest przejście automatu. Pozwala to generować słowa w kolejności rosnącej długości oraz automatycznie budować zestaw zweryfikowanych przykładów wykorzystywanych następnie przez model językowy. Dzięki temu model otrzymuje rzeczywiste przykłady działania automatu zamiast generować je wyłącznie na podstawie własnej wiedzy.

Algorytm BFS wykorzystywany jest również podczas analizy automatów ze stosem (PDA). W tym przypadku przeszukiwany graf nie jest jawnie zapisany w pamięci, lecz tworzony dynamicznie podczas działania programu. Każdy wierzchołek odpowiada konfiguracji automatu opisanej trójką

$$(q, i, S), \quad (69)$$

gdzie  $q$  oznacza aktualny stan automatu,  $i$  pozycję analizowanego symbolu wejściowego, natomiast  $S$  zawartość stosu. Kolejne konfiguracje generowane są na podstawie możliwych przejść automatu. Akceptacja następuje po osiągnięciu końca słowa wejściowego oraz spełnieniu warunku akceptacji zdefiniowanego dla automatu. Takie podejście pozwala jednocześnie eksplorować wszystkie możliwe ścieżki działania niedeterministycznego automatu ze stosem.

Kolejnym zastosowaniem BFS jest generowanie zbiorów testowych wykorzystywanych podczas benchmarków oraz analizy automatów ze stosem. W tym przypadku wierzchołkami grafu są wszystkie słowa nad danym alfabetem, natomiast krawędzie odpowiadają operacji dopisania kolejnego symbolu. Przeszukiwanie wszerz gwarantuje generowanie słów w kolejności niemalejącej długości, dzięki czemu krótsze słowa są analizowane przed dłuższymi. Pozwala to uzyskać reprezentatywny zbiór przypadków testowych przy ograniczonym koszcie obliczeniowym.

### **3.8.2 Przeszukiwanie wszerz w odwróconym grafie**

Istotnym elementem analizy automatów skończonych jest identyfikacja stanów pułapek. W tym celu platforma wykorzystuje przeszukiwanie wszerz wykonywane na odwróconym grafie przejść.

Najpierw odwracane są wszystkie krawędzie automatu, a następnie algorytm BFS rozpoczyna działanie jednocześnie ze wszystkich stanów akceptujących. Odwiedzone zostają wszystkie stany, z których istnieje ścieżka prowadząca do akceptacji. Stany osiągalne ze stanu początkowego, które nie zostały odwiedzone podczas tego przeszukiwania, klasyfikowane są jako stany pułapki. Oznacza to, że po wejściu do takiego stanu nie istnieje już możliwość zaakceptowania żadnego słowa. Informacja ta jest wykorzystywana podczas automatycznej analizy właściwości rozpoznawanego języka.

### **3.8.3 Przeszukiwanie w głąb (Depth-First Search)**

Drugim podstawowym algorytmem wykorzystywanym w platformie jest przeszukiwanie w głąb (Depth-First Search – DFS). W przeciwieństwie do BFS algorytm rozwija ścieżkę tak długo, jak jest to możliwe, zanim powróci do wcześniejszych wierzchołków [11].

W opracowanej platformie DFS wykorzystywany jest przede wszystkim do wykrywania cykli w grafie przejść automatu. Implementacja wykorzystuje klasyczny mechanizm kolorowania wierzchołków trzema kolorami: białym oznaczającym wierzchołki nieodwiedzone, szarym oznaczającym wierzchołki aktualnie przetwarzane oraz czarnym oznaczającym zakończone przeszukiwanie.

Pojawienie się krawędzi prowadzącej do wierzchołka oznaczonego kolorem szarym oznacza wykrycie cyklu w grafie przejść. Jeżeli cykl znajduje się w części grafu osiągalnej ze stanu początkowego oraz prowadzącej do stanu akceptującego, automat może akceptować nieskończenie wiele słów. W przeciwnym przypadku język rozpoznawany przez automat pozostaje skończony. Informacja ta stanowi jeden z elementów automatycznej charakterystyki języka generowanej przez platformę.

Połączenie algorytmów BFS i DFS umożliwia kompleksową analizę własności automatów skończonych oraz automatów ze stosem. Algorytmy te współpracują z modułami wykorzystującymi modele językowe, dostarczając zweryfikowanych informacji o strukturze automatu, przykładowych słów oraz właściwościach rozpoznawanego języka. Dzięki temu modele sztucznej inteligencji nie muszą samodzielnie wnioskować o zachowaniu automatu,

lecz korzystają z danych uzyskanych metodami algorytmicznymi, co zwiększa poprawność i wiarygodność generowanych analiz.

## Rozdział 4. Projekt i działanie aplikacji

Celem opracowanej aplikacji było stworzenie platformy umożliwiającej analizę, generowanie oraz wizualizację struktur języków formalnych z wykorzystaniem metod sztucznej inteligencji. System został zaprojektowany jako aplikacja webowa pozwalająca na interaktywną pracę z automatami skończonymi, automatami ze stosem oraz gramatykami bezkontekstowymi. Istotnym założeniem projektu było połączenie klasycznych metod teorii automatów i języków formalnych z możliwościami współczesnych modeli językowych.

Aplikacja umożliwia zarówno ręczne konstruowanie struktur formalnych, jak i ich automatyczne generowanie na podstawie opisów zapisanych w języku naturalnym. Oprócz funkcji generacyjnych system oferuje również rozbudowane mechanizmy analizy, pozwalające między innymi na badanie własności automatów, sprawdzanie przynależności słów do języka oraz wizualizację działania wybranych algorytmów.

### 4.1. Architektura systemu

[23, 24]

Platforma została zaimplementowana z wykorzystaniem technologii ASP.NET Core 8 zgodnie z architekturą MVC (Model–View–Controller). Takie podejście pozwala na wyraźne rozdzielenie warstwy prezentacji, logiki biznesowej oraz modeli danych, co zwiększa czytelność kodu i ułatwia dalszy rozwój aplikacji.

Warstwa prezentacji odpowiada za komunikację z użytkownikiem oraz wizualizację struktur formalnych. Została ona zrealizowana przy użyciu widoków Razor oraz języka JavaScript. Dzięki zastosowaniu technologii webowych użytkownik może w sposób interaktywny tworzyć automaty, definiować gramatyki oraz obserwować wyniki działania algorytmów.

Logika biznesowa została zaimplementowana w postaci zestawu usług odpowiedzialnych za analizę, generowanie oraz przetwarzanie struktur formalnych. Poszczególne komponenty realizują określone zadania, takie jak analiza automatów skończonych, analiza automatów ze stosem, obsługa algorytmu  $L^*$ , konwersja gramatyk do postaci normalnej Chomsky’ego czy wykonywanie algorytmu CYK.

Warstwa danych odpowiada za przechowywanie aktualnie tworzonych struktur formalnych. W projekcie zastosowano mechanizm sesji użytkownika, dzięki któremu możliwe jest zachowanie stanu edytowanych automatów i gramatyk pomiędzy kolejnymi żądaniami HTTP.

## 4.2. Model danych

W aplikacji zdefiniowano modele reprezentujące podstawowe struktury występujące w teorii języków formalnych. Najważniejszym z nich jest model automatu skończonego składający się ze zbioru stanów, alfabetu wejściowego, funkcji przejścia, stanu początkowego oraz zbioru stanów akceptujących. Poszczególne stany reprezentowane są przez obiekty klasy **State**, natomiast przejścia przez obiekty klasy **Transition**.

Drugim istotnym modelem jest automat ze stosem. Oprócz elementów charakterystycznych dla automatów skończonych zawiera on również alfabet stosu, symbol początkowy stosu oraz zbiór operacji wykonywanych na stosie podczas realizacji przejść.

System umożliwia również definiowanie gramatyk bezkontekstowych. Gramatyka przechowywana jest w postaci zbioru produkcji oraz symbolu startowego. Taka reprezentacja pozwala na wykonywanie dalszych operacji analitycznych, takich jak konwersja do postaci normalnej Chomsky'ego czy sprawdzanie przynależności słów do języka.

```

1 public class FiniteAutomaton
2 {
3     public string Name { get; set; } = "New_Automaton";
4     public FormalStructureType StructureType { get; set; } =
5         FormalStructureType.DeterministicFiniteAutomaton;
6
7     public List<State> States { get; set; } = new();
8     public List<string> Alphabet { get; set; } = new();
9     public List<Transition> Transitions { get; set; } = new();
10
11     public string StartState =>
12         States.FirstOrDefault(x => x.IsStart)?.Name ?? string.
13             Empty;
14
15     public List<string> AcceptingStates =>
16         States.Where(x => x.IsAccepting)
17             .Select(x => x.Name)
18             .ToList();
19
20     public GenerationInfo? GenerationInfo { get; set; }
21 }

```

Listing 1: Model automatu skończonego wykorzystany w aplikacji

Model ten bezpośrednio odwzorowuje podstawowe elementy automatu skończonego. Lista **States** przechowuje stany konstrukcji, **Alphabet** alfabet wejściowy, natomiast **Transitions** zbiór przejść. Właściwości **StartState** oraz **AcceptingStates** pozwalają na wygodne uzyskanie informacji o stanie początkowym i stanach akceptujących.

### 4.3. Moduł generowania struktur z wykorzystaniem sztucznej inteligencji

Jednym z najważniejszych elementów platformy jest moduł wykorzystujący modele językowe do generowania struktur formalnych. Zastosowane rozwiązanie umożliwia użytkownikowi opisanie języka w postaci naturalnego tekstu, który następnie interpretowany jest przez model sztucznej inteligencji.

Proces generowania rozpoczyna się od przekazania opisu języka do modelu językowego uruchamianego lokalnie za pomocą środowiska Ollama. Model analizuje otrzymane polecenie i generuje reprezentację struktury formalnej odpowiadającej podanemu opisowi. Następnie wygenerowane dane są przekształcane do wewnętrznych modeli wykorzystywanych przez aplikację, po czym prezentowane użytkownikowi w postaci graficznej.

Takie podejście pozwala znacząco uprościć proces tworzenia automatów i gramatyk, szczególnie dla użytkowników posiadających ograniczoną wiedzę z zakresu teorii języków for-

malnych.

```

1 public async Task<bool> MembershipQuery(string word)
2 {
3     if (_cache.TryGetValue(word, out var cached))
4     {
5         RawResponses.Add(
6             $" [{word}] -> (cache: {(cached ? "YES" : "NO")}");
7         return cached;
8     }
9
10    var wordDisplay = word == ""
11        ? "empty_word"
12        : $" '{word}'";
13
14    var prompt = $"Language_description: {_description}
15    Analyzed_word: {wordDisplay}
16
17    Does this word belong to the language?
18    Answer ONLY: YES or NO.";
19
20    int yesVotes = 0;
21
22    for (int i = 0; i < _votes; i++)
23    {
24        var result = await _ollama.AskAsync(prompt);
25        var parsed = ParseAnswer(result);
26
27        if (parsed == true)
28            yesVotes++;
29    }
30
31    bool answer = yesVotes * 2 > _votes;
32
33    _cache[word] = answer;
34    return answer;
35 }

```

Listing 2: Realizacja zapytania o przynależność słowa z wykorzystaniem modelu językowego

Przedstawiony fragment pokazuje sposób wykorzystania modelu językowego w charakterze wyroczni odpowiadającej na zapytania o przynależność słów. Dla każdego nowego słowa generowane jest zapytanie zawierające opis analizowanego języka. W celu ograniczenia wpływu pojedynczej błędnej odpowiedzi model może zostać zapytany wielokrotnie, a wynik ustalany jest na podstawie głosowania większościowego. Wyniki zapytań są do-

datkowo zapamiętywane w pamięci podręcznej.

```

1  var response =
2      await _ollama.AskAsync(
3          BuildRegexPrompt(description, alphabet));
4
5  var candidateRegex =
6      ExtractRegexLine(response);
7
8  RegexNode ast;
9
10 try
11 {
12     ast = new RegexParser(
13         candidateRegex, alphabet).Parse();
14 }
15 catch (RegexParseException ex)
16 {
17     Candidates.Add(new RegexCandidateInfo
18     {
19         Regex = candidateRegex,
20         ParseError = ex.Message
21     });
22
23     continue;
24 }
25
26 var nfa =
27     _thompson.Build(ast, alphabet);
28
29 var dfa =
30     _determinizer.Convert(nfa, alphabet);
31
32 DfaCompletion.CompleteWithTrapState(
33     dfa, alphabet);
34
35 var minimal =
36     _minimizer.Minimize(dfa, alphabet);

```

Listing 3: Hybrydowy proces generowania automatu DFA na podstawie wyrażenia regularnego

#### 4.4. Implementacja algorytmu L\*

[8]

W systemie zaimplementowano algorytm aktywnego uczenia automatów  $L^*$  zaproponowany przez Danę Angluin. Algorytm ten umożliwia automatyczne konstruowanie deterministycznego automatu skończonego na podstawie odpowiedzi udzielanych przez nauczyciela (ang. *oracle*).

Podstawą działania algorytmu jest tabela obserwacji zawierająca informacje o przynależności wybranych słów do analizowanego języka. W kolejnych iteracjach tabela jest rozszerzana do momentu spełnienia warunków domknięcia i spójności. Na podstawie uzyskanych danych tworzona jest hipoteza automatu reprezentująca aktualną wiedzę o języku.

W opracowanej platformie rolę nauczyciela może pełnić model językowy, który odpowiada na zapytania dotyczące przynależności słów do języka. Pozwala to na eksperymentalne połączenie klasycznych metod aktywnego uczenia z nowoczesnymi technikami sztucznej inteligencji.

```

1 public async Task<FiniteAutomaton> LearnAsync(
2     IAutomatonOracle oracle,
3     List<string> alphabet)
4 {
5     var S = new List<string> { "" };
6     var E = new List<string> { "" };
7     E.AddRange(alphabet);
8
9     var table =
10         new Dictionary<(string, string), bool>();
11
12     await FillTable(
13         oracle, S, E, alphabet, table);
14
15     await MakeClosedAndConsistent(
16         oracle, S, E, alphabet, table);
17
18     var hypothesis =
19         BuildAutomaton(S, E, alphabet, table);
20
21     for (int round = 0;
22         round < MaxEquivalenceRounds; round++)
23     {
24         var counterexample =
25             await FindCounterexampleAsync(
26                 oracle, hypothesis,
27                 alphabet);
28
29         if (counterexample == null)
30             return hypothesis;
31
32         foreach (var prefix
33             in AllPrefixes(counterexample))
34         {
35             if (!S.Contains(prefix))
36                 S.Add(prefix);
37         }
38
39         await FillTable(
40             oracle, S, E, alphabet, table);
41
42         await MakeClosedAndConsistent(
43             oracle, S, E, alphabet, table);

```

```
44
45     hypothesis =
46         BuildAutomaton(
47             S, E, alphabet, table);
48     }
49
50     return hypothesis;
51 }
```

Listing 4: Kluczowy fragment implementacji algorytmu  $L^*$

```

1 private async Task MakeClosedAndConsistent(
2     IAutomatonOracle oracle,
3     List<string> S,
4     List<string> E,
5     List<string> alphabet,
6     Dictionary<(string, string), bool> table)
7 {
8     bool changed;
9     do
10    {
11        changed = false;
12        var violation =
13            FindClosednessViolation(
14                S, alphabet, E, table);
15        if (violation != null)
16        {
17            S.Add(violation);
18            await FillTable(
19                oracle, S, E,
20                alphabet, table);
21
22            changed = true;
23            continue;
24        }
25        var suffix = FindConsistencyViolation( S, E, alphabet,
26            table);
27        if (suffix != null)
28        {
29            E.Add(suffix);
30            await FillTable(
31                oracle, S, E,
32                alphabet, table);
33            changed = true;
34        }
35    } while (changed);
36 }

```

Listing 5: Sprawdzanie domknięcia i spójności tabeli obserwacji

```

1 private FiniteAutomaton BuildAutomaton(List<string> S,
2     List<string> E, List<string> alphabet,
3     Dictionary<(string, string), bool> table){
4     string RowSig(string s) => RowSignature(s, E, table);
5     var representatives =
6     S.GroupBy(RowSig).Select(g => g.First()).ToList();
7     var stateNames = representatives.ToDictionary(r => RowSig(r),
8         r => $"q{representatives.IndexOf(r)}");
9     var automaton = new FiniteAutomaton{
10         Name = "L*_Generated_DFA", Alphabet = alphabet,
11         States = new List<State>(),
12         Transitions = new List<Transition>()
13     };
14     foreach (var rep in representatives){
15         var sig = RowSig(rep);
16         automaton.States.Add(new State{
17             Name = stateNames[sig],
18             IsStart = rep == "",
19             IsAccepting = table.GetValueOrDefault((rep, ""))
20         });
21     }
22     foreach (var rep in representatives){
23         foreach (var symbol in alphabet){
24             var successor = rep + symbol;
25             var sig = RowSig(successor);
26             if (stateNames.TryGetValue(sig, out var target)){
27                 automaton.Transitions.Add(new Transition{
28                     FromState = stateNames[RowSig(rep)],
29                     Symbol = symbol, ToState = target
30                 });
31             }
32         }
33     }
34     return automaton;
35 }

```

Listing 6: Konstrukcja automatu DFA na podstawie tabeli obserwacji

## 4.5. Moduł analizy automatów skończonych

Aplikacja zawiera moduł umożliwiający automatyczną analizę automatów skończonych. Podczas analizy wyznaczane są podstawowe parametry konstrukcji, takie jak liczba stanów, liczba przejść, wielkość alfabetu czy liczba stanów akceptujących.

Oprócz prostych statystyk system przeprowadza również analizę strukturalną automatu. W szczególności sprawdzana jest deterministyczność konstrukcji oraz poprawność zdefiniowanych przejść. Wyniki prezentowane są użytkownikowi w postaci raportu tekstowego.

```

1 public LanguageAnalysisResult Analyze(
2     FiniteAutomaton automaton)
3 {
4     var result =
5         new LanguageAnalysisResult();
6
7     var stateNames =
8         automaton.States
9             .Select(s => s.Name)
10            .ToHashSet();
11
12     var acceptingStates =
13         automaton.States
14             .Where(s => s.IsAccepting)
15             .Select(s => s.Name)
16             .ToHashSet();
17
18     var alphabet =
19         automaton.Alphabet;
20
21     result.IsComplete =
22         automaton.States.All(state =>
23             alphabet.All(symbol =>
24                 automaton.Transitions.Any(t =>
25                     t.FromState == state.Name &&
26                     t.Symbol == symbol))));
27
28     result.ReachableStates =
29         GetReachableStates(
30             automaton.StartState,
31             automaton.Transitions);
32
33     result.UnreachableStates =
34         stateNames
35             .Except(result.ReachableStates)
36             .ToList();
37     return result;
38 }

```

Listing 7: Przygotowanie danych automatu do analizy języka

## 4.6. Moduł analizy automatów ze stosem

W przypadku automatów ze stosem zaimplementowano bardziej zaawansowane metody analizy wynikające z większej złożoności tych struktur. System umożliwia symulację działania automatu dla wybranego słowa wejściowego, śledząc jednocześnie zawartość stosu na kolejnych etapach obliczeń.

Dodatkowo realizowana jest analiza osiągalności stanów oraz wyszukiwanie przykładowych słów akceptowanych i odrzucanych przez automat. Do eksploracji przestrzeni konfiguracji wykorzystano algorytm przeszukiwania wszerz (BFS), który pozwala efektywnie analizować możliwe ścieżki wykonania.

```

1 var initial = new Configuration
2 {
3     State = automaton.StartState,
4     Position = 0,
5     Stack = new Stack<string>()
6 };
7
8 var queue = new Queue<Configuration>();
9 var visited = new HashSet<string>();
10
11 queue.Enqueue(initial);
12
13 while (queue.Count > 0)
14 {
15     var current = queue.Dequeue();
16     var key = CreateConfigurationKey(current);
17
18     if (!visited.Add(key))
19         continue;
20
21     if (current.Position == word.Length &&
22         IsAccepting(current))
23         return true;
24
25     foreach (var transition
26         in GetAvailableTransitions(current))
27     {
28         var next = ApplyTransition(
29             current, transition, word);
30
31         if (next != null)
32             queue.Enqueue(next);
33     }
34 }
35 return false;

```

Listing 8: Symulacja automatu ze stosem z wykorzystaniem BFS

## 4.7. Obsługa gramatyk bezkontekstowych

Platforma udostępnia rozbudowany moduł pracy z gramatykami bezkontekstowymi. Użytkownik może definiować własne produkcje, analizować strukturę gramatyki oraz wykonywać operacje transformacji.

Szczególnie istotnym elementem jest możliwość konwersji gramatyki do postaci normalnej Chomsky’ego, która stanowi podstawę działania algorytmu CYK. Dodatkowo zaimplementowano procedurę konwersji gramatyki bezkontekstowej do równoważnego automatu ze stosem, co pozwala na prezentację zależności pomiędzy różnymi modelami obliczeń.

```

1  var pda = new PushdownAutomaton
2  {
3      States = new List<State>(),
4      Transitions = new List<PdaTransition>(),
5      InputAlphabet = grammar.Terminals,
6      StackAlphabet = grammar.NonTerminals
7  };
8  var startState = new State{ Name = "q0", IsStart = true };
9  var loopState = new State { Name = "q1" };
10 pda.States.Add(startState);
11 pda.States.Add(loopState);
12 foreach (var production in grammar.Productions)
13 {
14     pda.Transitions.Add(
15         new PdaTransition
16         {
17             FromState = loopState.Name,
18             ToState = loopState.Name,
19             InputSymbol = null,
20             StackTop = production.Left,
21             PushSymbols =
22                 production.Right
23         });
24 }
25 foreach (var terminal in grammar.Terminals)
26 {
27     pda.Transitions.Add(
28         new PdaTransition
29         {
30             FromState = loopState.Name,
31             ToState = loopState.Name,
32             InputSymbol = terminal,
33             StackTop = terminal,
34             PushSymbols =
35                 new List<string>()
36         });
37 }

```

Listing 9: Konstrukcja automatu ze stosem na podstawie gramatyki bezkontekstowej

## 4.8. Implementacja algorytmu CYK

[9, 10]

W celu sprawdzania przynależności słów do języków generowanych przez gramatyki bezkontekstowe zaimplementowano algorytm Cocke’a–Youngera–Kasamiego (CYK). Algorytm ten działa na gramatykach zapisanych w postaci normalnej Chomsky’ego i wykorzystuje programowanie dynamiczne.

Podczas działania tworzona jest trójkątna tablica zawierająca informacje o nieterminalach zdolnych do wygenerowania poszczególnych fragmentów analizowanego słowa. Wypełnianie tablicy rozpoczyna się od symboli terminalnych, a następnie stopniowo analizowane są coraz dłuższe fragmenty wejścia. Ostatecznie obecność symbolu startowego w ostatniej komórce tablicy oznacza, że badane słowo należy do języka generowanego przez gramatykę.

```

1  int n = word.Length;
2  var table = new HashSet<string>[n, n];
3  for (int i = 0; i < n; i++)
4      for (int j = 0; j < n; j++)
5          table[i, j] = new HashSet<string>();
6
7  for (int i = 0; i < n; i++)
8  {
9      foreach (var production
10         in grammar.TerminalProductions)
11      {
12          if (production.Terminal == word[i])
13              table[i, i].Add(production.Left);
14      }
15  }
16  for (int len = 2; len <= n; len++)
17  {
18      for (int i = 0; i <= n - len; i++)
19      {
20          int j = i + len - 1;
21          for (int k = i; k < j; k++)
22          {
23              foreach (var production
24                 in grammar.BinaryProductions)
25              {
26                  if (table[i, k].Contains(
27                      production.LeftSymbol) &&
28                      table[k + 1, j].Contains(
29                          production.RightSymbol))
30                  {
31                      table[i, j].Add(
32                          production.Left);
33                  }
34              }
35          }
36      }
37  }
38  return table[0, n - 1].Contains(grammar.StartSymbol);

```

Listing 10: Wypełnianie tablicy programowania dynamicznego w algorytmie CYK

## 4.9. Graficzny edytor struktur formalnych

Integralnym elementem systemu jest interaktywny edytor grafowy umożliwiający wizualne tworzenie struktur formalnych. Użytkownik może dodawać nowe stany, tworzyć przejścia pomiędzy nimi oraz oznaczać stany początkowe i akceptujące.

Wszystkie zmiany są natychmiast prezentowane na ekranie, co znacząco ułatwia projektowanie i analizowanie automatów. Zastosowanie interfejsu graficznego eliminuje konieczność ręcznego definiowania struktur w postaci matematycznej, dzięki czemu korzystanie z systemu staje się bardziej intuicyjne.

```

1 public void AddState(
2     FiniteAutomaton automaton,
3     string stateName,
4     bool isStart,
5     bool isAccepting,
6     double x,
7     double y)
8 {
9     if (string.IsNullOrEmpty(stateName))
10         return;
11
12     if (automaton.States.Any(
13         s => s.Name == stateName))
14         return;
15
16     if (isStart)
17     {
18         foreach (var state in
19             automaton.States)
20         {
21             state.IsStart = false;
22         }
23     }
24
25     automaton.States.Add(
26         new State
27         {
28             Name = stateName,
29             IsStart = isStart,
30             IsAccepting = isAccepting,
31             X = x,
32             Y = y
33         });
34 }

```

Listing 11: Dodawanie stanu w graficznym edytorze automatu

## Rozdział 5. Analiza rozwiązania

### 5.1. Metodologia ewaluacji

Do oceny skuteczności platformy zbudowano skrypt `benchmark.py`, porównujący trzy warianty generowania struktur formalnych z opisu w języku naturalnym:

- **ChatGPT-4o** - pojedyncze zapytanie do modelu komercyjnego, odpowiedź parsowana z tekstu;
- **Ollama (raw)** - pojedyncze zapytanie do lokalnego modelu (Mistral-7B [19]) bez żadnej warstwy formalnej, odpowiedź parsowana analogicznie;
- **FSP (hybryda)** - pełny pipeline platformy:  $L^*$  z wyrocznią LLM dla automatów regularnych (z jednostrzałową ścieżką regex jako szybszą alternatywą) oraz generowanie gramatyki CFG i konstrukcja PDA dla języków bezkontekstowych.

Zbiór testowy obejmuje 12 języków w trzech kategoriach: *trywialne* (*ends\_b*, *contains\_a*, *starts\_ab*), *regularne* (*even\_a*, *div3\_binary*, *no\_aa*, *ends\_ab*, *len\_mod3*) oraz *bezkontekstowe* (*anbn*, *balanced\_parens*, *palindrome\_ab*, *equal\_ab*). Dla każdego języka generowany automat/gramatyka jest symulowany na wszystkich słowach testowych do zadanej długości (127 słów dla języków regularnych, 255 - 511 dla bezkontekstowych), a *accuracy* to odsetek poprawnie sklasyfikowanych słów. Dodatkowo rejestrowane są: powodzenie parsowania odpowiedzi modelu, kompletność wygenerowanej struktury (czy symulacja daje jednoznaczną odpowiedź dla każdego słowa) oraz czas generowania.

## 5.2. Iteracyjne poprawki algorytmu $L^*$

Wstępne przebiegi benchmarku ujawniły, że wariant FSP dla języków regularnych bywał gorszy od naiwnego zapytania do surowego modelu - sygnał, że problem leży w implementacji, nie w samej koncepcji hybrydowej. Analiza kodu i wyników doprowadziła do zidentyfikowania i naprawienia następujących błędów:

- **Brak sprawdzania spójności tabeli obserwacji** - implementacja  $L^*$  weryfikowała wyłącznie domkniętość (*closedness*), pomijając sprawdzanie spójności (*consistency*), co mogło prowadzić do błędnego scalania faktycznie różnych stanów. Dodano pełne sprawdzanie spójności zgodnie z klasycznym algorytmem Angluina.
- **Brak pętli zapytań o równoważność** - pierwsza domknięta tabela była akceptowana bez żadnej weryfikacji względem wyroczni, co przy zaszumionych odpowiedziach LLM prowadziło do automatów z rażąco zaniżoną liczbą stanów (zaobserwowano automat o 1 stanie dla języka opartego o długość modulo 3, podczas gdy minimalna liczba stanów wynosi 3). Dodano pętlę *equivalence query* z przetwarzaniem kontrprzykładu, testującą hipotezę na dodatkowych słowach przed jej zaakceptowaniem.
- **Szum wyroczni LLM** - wprowadzono głosowanie większościowe (3-krotne zapytanie o to samo słowo) w miejscach krytycznych dla poprawności strukturalnej, redukujące wpływ pojedynczych, przypadkowych pomyłek modelu.

- **Niekompletność automatów** - automaty zwracane z tabeli, która nie zdążyła się w pełni domknąć (np. z powodu budżetu czasowego), mogły mieć brakujące przejścia. Dodano jawne dopełnianie stanem pułapką, gwarantujące całkowitość zwracanego DFA niezależnie od tego, na jakim etapie nauka została przerwana.
- **Sprzeczność w promptcie generowania gramatyki** - dla języka `balanced_parens` (alfabet: nawiasy) prompt jednocześnie podawał właściwy alfabet i instruował model, by terminale ograniczał do liter/cyfr. Sprzeczność powodowała sporadyczne, drastyczne spadki accuracy (do 0.05) przy niezmiennym kodzie. Poprawiono prompt, by dynamicznie opisywał rzeczywisty alfabet zamiast zakładać litery/cyfry.

Dodatkowo zaimplementowano alternatywną, jednostrzałową ścieżkę generowania automatów regularnych, analogiczną do już sprawdzonej architektury CFG: model generuje wyrażenie regularne *raz*, a klasyczne algorytmy formalne (konstrukcja Thompsona, determinizacja, minimalizacja metodą Moore’a) budują z niego DFA. Wygenerowany kandydat przechodzi samospójnościową weryfikację (3 niezależne generacje, wybór najlepszej względem próbki słów ocenionej przez wyrocznie z głosowaniem większościowym) i w razie niepowodzenia system spada do pełnego  $L^*$  jako mechanizmu awaryjnego.

### 5.3. Wyniki końcowe dla modelu bazowego

Tabela 1 przedstawia reprezentatywne wyniki accuracy po wdrożeniu wszystkich powyższych poprawek, dla modelu bazowego Mistral-7B. Ze względu na niedeterminizm generowania (opisany w podrozdziale 5.4), podano zakresy wartości zaobserwowane w kilku niezależnych przebiegach identycznego kodu.

| Język                 | Ollama (raw) | FSP (hybryda) |
|-----------------------|--------------|---------------|
| <i>Trywialne</i>      |              |               |
| ends_b                | 0.50–1.00    | 0.86–1.00     |
| contains_a            | 0.46         | 0.71–1.00     |
| starts_ab             | 0.51         | 0.36–0.57     |
| <i>Regularne</i>      |              |               |
| even_a                | 0.50         | 0.49–0.58     |
| div3_binary           | 0.00–0.48    | 0.35–0.64     |
| no_aa                 | 0.41–0.59    | 0.43–0.80     |
| ends_ab               | 0.24–0.75    | 0.25–0.76     |
| len_mod3              | 0.42–0.57    | 0.44–0.68     |
| <i>Bezkontekstowe</i> |              |               |
| anbn                  | 0.00         | 0.87–0.99     |
| balanced_parens       | 0.00         | 0.93–0.95     |
| palindrome_ab         | 0.00         | 0.82–0.83     |
| equal_ab              | 0.00         | 0.81–0.87     |

Tabela 1: Accuracy klasyfikacji słów testowych po wdrożeniu poprawek (Mistral-7B jako model bazowy).

Wyniki potwierdzają główną tezę pracy w sposób zróżnicowany między kategoriami języków. Dla języków bezkontekstowych przewaga architektury hybrydowej jest jednoznaczna i stabilna. FSP osiąga 0.81 - 0.99 accuracy, podczas gdy surowy model uzyskuje 0.00 trafności dla każdego z czterech testowanych języków (parsowanie odpowiedzi konsekwentnie się nie powodzi - modele nie potrafią poprawnie opisać struktury wymagającej pamięci stosowej w pojedynczej odpowiedzi tekstowej). Dla języków regularnych obraz jest bardziej niejednoznaczny: FSP przewyższa surowy model w większości przypadków, ale przewaga jest znacznie mniejsza i w kilku językach (`starts_ab`, częściowo `div3_binary`) architektura hybrydowa nie przynosi wyraźnej korzyści względem prostego zapytania.

## 5.4. Wariancja między przebiegami

Wielokrotne uruchomienia identycznego kodu na tym samym modelu dały zauważalnie różne wyniki - np. `ends_b` wahało się między 0.76 a 1.00, a liczba stanów wyuczona dla `len_mod3` oscylowała między 1 (błędnie, przed poprawkami) a 3 (poprawnie, teoretyczne minimum) w zależności od przebiegu, mimo braku zmian w kodzie między uruchomieniami. Źródłem tej wariancji jest niedeterminizm samego modelu językowego (niezerowa temperatura generowania) w połączeniu z tym, że zarówno  $L^*$ , jak i ścieżka regexowa, opierają swoją poprawność na wielokrotnych, niezależnych zapytaniach do tego samego, niedoskonałego źródła.

## 5.5. Eksperyment: doszkalanie modelu metodą LoRA

[18]

Mimo wdrożenia opisanych powyżej poprawek algorytmicznych, pewne kategorie języków - w szczególności wymagające rozpoznawania własności modularnych (`div3_binary`, `len_mod3`) oraz rozróżniania pozycji wzorca w słowie (`starts_ab` vs. `contains_a`) - pozostały systematycznie słabe niezależnie od dalszych modyfikacji kodu. Powtarzalność tego wzorca w kolejnych, niezależnych poprawkach sugerowała, że przyczyną nie jest błąd implementacyjny, lecz rzeczywiste ograniczenie kompetencji modelu bazowego w tych konkretnych zadaniach. Zdecydowano się zweryfikować tę hipotezę eksperymentalnie poprzez doszkalanie (*fine-tuning*) metodą LoRA.

### 5.5.1 Generowanie danych treningowych

Kluczowym założeniem metodologicznym było wygenerowanie danych treningowych w sposób w pełni syntetyczny i formalnie zweryfikowany, bez ręcznego etykietowania. Zbudowano generator obejmujący:

- **Sparametryzowane rodziny języków**: wzorce podłańcuchowe (zaczyna się/kończy się/zawiera/nie zawiera, które są budowane klasycznym automatem KMP), długość słowa modulo  $k$ , liczba wystąpień symbolu modulo  $k$ , oraz wartość liczby w zapisie pozycyjnym o zadanej podstawie modulo  $k$  (uogólnienie `div3_binary`);
- **Konwerter DFA→regex** metodą eliminacji stanów (algorytm McNaughtona–Yamady), generujący docelowe wyrażenia regularne w sposób w pełni deterministyczny, bez udziału LLM;
- **Niezależny validator** - osobna implementacja parsera wyrażeń regularnych wraz z symulacją metodą wyprowadzeń Brzozowskiego, używana wyłącznie do potwierdzenia, że każda wygenerowana para (opis, regex) jest semantycznie poprawna względem oryginalnego DFA, zanim trafi do zbioru treningowego.

Podział na zbiór treningowy i testowy przeprowadzono *po parametrach szablonu*, nie po pojedynczych przykładach: modulo 3 (zarówno dla długości słowa, jak i dla wartości w zapisie binarnym) oraz wzorzec "ab" zostały celowo wykluczone z treningu i umieszczone wyłącznie w zbiorze testowym. Taki podział odtwarza dokładnie sytuację najslabszych kategorii z benchmarku (`len_mod3`, `div3_binary`, `starts_ab/ends_ab`), a jednocześnie pozwala zmierzyć rzeczywiste *uogólnienie* modelu na nieznaną kombinację parametrów, zamiast zapamiętania konkretnych przykładów.

Ostatecznie uzyskano ok. 8490 przykładów dla zadania wyroczni (zapytanie o przynależność słowa: TAK/NIE) oraz 250 przykładów dla zadania generowania regexu (po odfiltrowaniu instancji, dla których algorytm eliminacji stanów wygenerował patologicznie długie wyrażenia - do ok. 22 tys. znaków dla języków modularnych o wyższych podstawach, co

jest znanym efektem ubocznym tej klasycznej metody i czyni takie przykłady praktycznie niemożliwymi do nauczenia).

### 5.5.2 Proces treningu i napotkane trudności

Doszkalanie przeprowadzono metodą QLoRA (kwantyzacja 4-bit, `rank=16`) na modelu Mistral-7B-v0.1, w środowisku Google Colab (GPU T4). Proces napotkał szereg trudności o charakterze infrastrukturalnym, które same w sobie stanowią wart odnotowania wnioski praktyczny dla podobnych eksperymentów:

- niekompatybilność precyzji `bfloat16` z architekturą GPU (Turing/T4, brak akceleracji sprzętowej), skutkująca spowolnieniem treningu o rząd wielkości, oraz błędem uniemożliwiającym trening przy równoczesnym użyciu `float16` z mechanizmem `GradScaler` bez jawnego, spójnego wymuszenia precyzji na wszystkich poziomach (model, adapter LoRA, konfiguracja `accelerate`);
- konflikty wersji bibliotek (`transformers`, `peft`, `trl`, `torchao`) charakterystyczne dla szybko zmieniającego się ekosystemu narzędzi do doszkalania modeli językowych;
- konieczność jawnego maskowania funkcji straty wyłącznie na części odpowiedzi (*completion-only loss*) - bez tego model uczy się przewidywać cały tekst promptu (silnie zdominowany przez powtarzalny szablon), a nie samą odpowiedź; w pierwszej próbie objawiło się to degeneracją modelu do generowania wyłącznie znaków nowej linii.

### 5.5.3 Wyniki i wnioski

Po naprawieniu maskowania straty i przeprowadzeniu treningu na pełnym zbiorze danych (ok. 6960 przykładów treningowych, 3 epoki), model doszkalany do roli wyroczni osiągnął **53,6% ogólnej zgodności** na zbalansowanym zbiorze testowym (765 przykładów TAK / 765 NIE). Rozbicie wyniku ze względu na kategorię ujawnia istotną strukturę:

- **59,6%** na kategoriach reprezentowanych (w innych wariantach parametrów) w zbiorze treningowym;
- **50,3%** - a więc dokładnie na poziomie przypadku - na module 3, celowo wykluczonym z treningu jako test rzeczywistego uogólnienia.

Model wykazywał ponadto silne, systematyczne obciążenie w stronę odpowiedzi przeczącej (92,7% predykcji 'NIE' przy zbalansowanym zbiorze testowym), co wskazuje na niedostateczną siłę sygnału treningowego przy zastosowanej konfiguracji (rank adaptera, liczba epok) do pełnego wyuczenia granicy decyzyjnej zależnej od treści opisu.

Eksperyment z doszkalaniem adaptera do generowania wyrażeń regularnych nie doprowadził do rozstrzygającej ewaluacji, po wyeliminowaniu problemu patologicznie długich celów treningowych oraz poprawieniu błędu metodologicznego w skrypcie ewaluacyjnym

(limit generowanych tokenów dostosowany do zadania wyroczni, 8 tokenów, okazał się drastycznie zbyt mały dla wielotokenowych wyrażeń regularnych, co sztucznie zaniżało zmierzoną skuteczność do 0%).

**Wniosek z eksperymentu:** doszkalanie przyniosło mierzalną, ale ograniczoną poprawę wyłącznie na kategoriach zbliżonych strukturalnie do widzianych w treningu. Wynik na module 3, dokładnie na poziomie przypadku mimo jego jawnej obecności (w innych wariantach) w zbiorze treningowym i mimo architektury podziału zaprojektowanej pod kątem promowania uogólnienia, stanowi mocny argument na rzecz hipotezy, że rozpoznawanie własności modularnych jest ograniczeniem zdolności rozumowania matematycznego małego modelu językowego w pojedynczym przebiegu obliczeniowym (bez jawnego łańcucha wnioskowania), a nie deficytem danych treningowych możliwym do usunięcia niewielkim doszkalanem LoRA. Obserwacja ta znajduje potwierdzenie w kolejnym eksperymencie, opisanym w podrozdziale 5.6.

## 5.6. Porównanie alternatywnych modeli bazowych

[19, 20, 21]

Jako alternatywę wobec doszkalania istotnie tańszą w realizacji przetestowano podmianę modelu bazowego obsługiwanego przez Ollamę, bez żadnych zmian w kodzie platformy poza konfiguracją. Wybrano dwa modele o odmiennej charakterystyce: **phi4-mini** [20], projektowany pod kątem zadań matematyczno-logicznych przy małym rozmiarze, oraz **DeepSeek-R1** (wariant 8B), model typu *reasoning*, generujący jawny, widoczny łańcuch rozumowania przed odpowiedzią końcową.

| Język           | Mistral-7B | phi4-mini | DeepSeek-R1-8B |
|-----------------|------------|-----------|----------------|
| ends_b          | 0.86–1.00  | 1.00      | 0.50*          |
| contains_a      | 0.71–1.00  | 0.71      | 0.06*          |
| starts_ab       | 0.36–0.57  | 0.41      | 0.76*          |
| even_a          | 0.49–0.58  | 0.58      | 0.50*          |
| div3_binary     | 0.35–0.64  | 0.56      | 0.64*          |
| no_aa           | 0.43–0.80  | 0.60      | 0.59*          |
| ends_ab         | 0.25–0.76  | 0.25      | 0.76*          |
| len_mod3        | 0.44–0.68  | 0.54      | 0.00†          |
| anbn            | 0.87–0.99  | 0.99      | 0.99           |
| balanced_parens | 0.93–0.95  | 0.95      | 0.95           |
| palindrome_ab   | 0.82–0.83  | 0.82      | 0.82           |
| equal_ab        | 0.81–0.87  | 0.87      | 0.87           |

Tabela 2: Porównanie accuracy dla trzech modeli bazowych. \* Automatyzacja dla DeepSeek-R1 w językach regularnych są zdegenerowane (1–2 stany) - budżet czasowy L\* wyczerpywał się po zaledwie 2–3 z 40 zaplanowanych zapytań ze względu na bardzo długi czas pojedynczej odpowiedzi modelu (rozumowanie krok-po-kroku); wysokie wartości accuracy w tej kolumnie są w dużej mierze artefaktem niezbalansowania klas w teście, nie efektem rzeczywistego uczenia się, i nie powinny być interpretowane jako przewaga modelu. † Przekroczono maksymalny, wydłużony limit czasu żądania (600s).

Wyniki dla języków bezkontekstowych są niemal identyczne dla wszystkich trzech modeli, mimo że pojedyncze zapytanie do DeepSeek-R1 trwało od kilkudziesięciu do ponad 500 sekund (wobec kilku sekund dla Mistrała). Ścieżka generatywno-weryfikacyjna okazała się w pełni odporna na tak drastyczną różnicę w szybkości i stylu odpowiedzi modelu bazowego. W przeciwną stronę zadziało to dla ścieżki L\*: ta sama różnica w szybkości modelu doprowadziła do praktycznej niefunkcjonalności tego komponentu przy DeepSeek-R1 - mechanizmy zabezpieczające przed nadmiernym zużyciem czasu (wprowadzone w toku prac, zob. podrozdział 5.2) zadziały zgodnie z założeniem, przerywając naukę przed przekroczeniem globalnego limitu czasu, lecz przy tak wolnym modelu zdążyły przetworzyć zaledwie ułamek zaplanowanych zapytań, co uniemożliwiło uzyskanie sensownego automatu.

Obserwacja ta stanowi niezależne, mocniejsze potwierdzenie wniosku architektonicznego sformułowanego już w kontekście eksperymentu z doszkalaniem: **architektura oparta na jednorazowej generacji z klasyczną weryfikacją formalną (zastosowana dla języków bezkontekstowych) jest odporna zarówno na jakość, jak i szybkość modelu bazowego, podczas gdy architektura oparta na wielokrotnym odpytywaniu LLM w roli wyroczni (algorytm L\* dla języków regularnych) jest wobec obu tych czynników krucha**. Spowolnienie lub obniżenie jakości modelu nie degraduje jej wyników stopniowo, lecz prowadzi do gwałtownej, jakościowej zmiany zachowania systemu.

## 5.7. Podsumowanie wniosków z analizy

Zebrane wyniki pozwalają zweryfikować postawioną we wstępie hipotezę badawczą w sposób zniuansowany. Dla języków bezkontekstowych hipoteza o przewadze podejścia hybrydowego nad czystym LLM znajduje jednoznaczne, powtarzalne potwierdzenie - zarówno w benchmarku porównawczym, jak i w eksperymencie z trzema różnymi modelami bazowymi. Dla języków regularnych obraz jest bardziej złożony: iteracyjne poprawki algorytmu L\* (sprawdzanie spójności, zapytania o równoważność, głosowanie większościowe, jednostrzałowa ścieżka regexowa) przyniosły wymierną poprawę względem początkowej, błędnej implementacji, lecz nie usunęły w pełni luki wobec prostego zapytania do modelu w kilku kategoriach, a w szczególności nie rozwiązały systematycznej słabości w zadaniach wymagających rozumowania modularnego - luki, której nie usunęło również ukierunkowane doszkalanie metodą LoRA. Sugeruje to, że o ile formalna weryfikacja i klasyczne algorytmy skutecznie korygują błędy strukturalne wynikające z niedoskonałości pojedynczych odpowiedzi modelu, o tyle nie są w stanie zrekompensować fundamentalnego braku kompetencji modelu bazowego w konkretnej klasie zadań, a to z kolei wskazuje architekturę minimalizującą powierzchnię, na jakiej niedoskonałość modelu językowego może wpływać na poprawność wyniku (jedno zapytanie generatywne plus weryfikacja klasyczna), jako kierunek o największym potencjale dalszego rozwoju platformy.

## Rozdział 6. Podsumowanie i wnioski

Celem niniejszej pracy było opracowanie platformy umożliwiającej analizę, generowanie oraz wizualizację struktur języków formalnych z wykorzystaniem metod sztucznej inteligencji, a także zbadanie możliwości wykorzystania dużych modeli językowych do transformacji opisów w języku naturalnym na formalne reprezentacje języków. Istotnym założeniem pracy było połączenie możliwości modeli językowych z klasycznymi algorytmami teorii automatów i języków formalnych, tak aby ograniczyć wpływ błędów generowanych przez modele i jednocześnie zachować możliwość automatycznego tworzenia struktur formalnych.

W ramach pracy przeanalizowano podstawowe zagadnienia związane z językami formalnymi, w tym gramatyki formalne, wyrażenia regularne, automaty skończone, automaty ze stosem oraz języki bezkontekstowe. Szczególną uwagę poświęcono metodom pozwalającym na automatyczną analizę i uczenie struktur formalnych, takim jak algorytm  $L^*$  Angluina oraz algorytm Cocke'a–Youngera–Kasamiego. Przedstawiono również możliwości zastosowania modeli językowych do zadań związanych z językami formalnymi oraz problemy wynikające z różnicy pomiędzy generowaniem tekstu w języku naturalnym a tworzeniem struktur podlegających ścisłym regułom formalnym.

W wyniku przeprowadzonych prac opracowano aplikację webową stanowiącą platformę eksperymentalną. System został zaimplementowany z wykorzystaniem technologii ASP.NET Core 8 oraz architektury MVC. Platforma umożliwia tworzenie i edycję automatów skończonych, automatów ze stosem oraz gramatyk bezkontekstowych, a także ich analizę i wizualizację. Zaimplementowano między innymi mechanizmy analizy automatów skończonych i automatów ze stosem, konwersję gramatyk bezkontekstowych do postaci normalnej Chomsky'ego, konwersję gramatyk do automatów ze stosem oraz algorytm CYK służący do sprawdzania przynależności słów do języka.

Istotną częścią platformy jest moduł wykorzystujący modele językowe uruchamiane lokalnie za pośrednictwem środowiska Ollama. Pozwala on na generowanie struktur formalnych na podstawie opisów przedstawionych w języku naturalnym. Dzięki temu użytkownik nie musi samodzielnie definiować wszystkich elementów automatu lub gramatyki w formalnym zapisie. Jednocześnie wygenerowane struktury mogą być poddawane dalszej analizie za pomocą klasycznych algorytmów, co pozwala oddzielić proces interpretacji opisu od procesu formalnej analizy utworzonej struktury.

Szczególnie istotnym elementem pracy było zastosowanie algorytmu  $L^*$  w połączeniu z modelem językowym pełniącym funkcję wyroczni. Takie rozwiązanie pozwoliło zbadać możliwość wykorzystania LLM do aktywnego uczenia automatów skończonych. Przeprowadzone eksperymenty wykazały jednak, że bezpośrednie połączenie algorytmu wymagającego wielu zapytań z niedeterministycznym modelem językowym jest podatne na błędy. W toku prac zidentyfikowano między innymi brak sprawdzania spójności tabeli obserwacji, brak pełnej obsługi zapytań o równoważność, wpływ pojedynczych błędnych odpowiedzi modelu oraz możliwość generowania niekompletnych automatów. Problemy te

zostały następnie wykorzystane do iteracyjnego udoskonalenia implementacji.

Wprowadzone poprawki obejmowały pełne sprawdzanie domknięcia i spójności tabeli obserwacji, wykorzystanie zapytań o równoważność wraz z kontrprzykładami, głosowanie większościowe dla wybranych zapytań oraz automatyczne uzupełnianie konstrukcji stanem pułapką. Dodatkowo opracowano alternatywną ścieżkę generowania automatów regularnych, w której model językowy generuje wyrażenie regularne w pojedynczym zapytaniu, natomiast dalsze tworzenie automatu realizowane jest za pomocą klasycznych algorytmów. Podejście to ogranicza liczbę miejsc, w których błędna odpowiedź modelu może bezpośrednio wpłynąć na wynik końcowy.

Przeprowadzone eksperymenty pozwoliły przede wszystkim porównać działanie samego modelu językowego z działaniem opracowanej architektury hybrydowej. Dla języków bezkontekstowych uzyskano wyraźną przewagę rozwiązania hybrydowego. W czterech badanych przypadkach model Mistral-7B wykorzystywany bez dodatkowej warstwy formalnej uzyskał skuteczność równą 0.00, podczas gdy platforma osiągnęła wartości od 0.81 do 0.99. Podobne wyniki uzyskano również po zastąpieniu modelu bazowego innymi modelami. Oznacza to, że w przypadku bardziej złożonych struktur formalnych klasyczne algorytmy stanowią skuteczną warstwę zabezpieczającą przed ograniczeniami pojedynczej odpowiedzi modelu językowego.

Znacznie bardziej złożone rezultaty uzyskano dla języków regularnych. W większości badanych przypadków rozwiązanie hybrydowe osiągało wyniki lepsze od surowego modelu, jednak różnica nie była tak wyraźna jak dla języków bezkontekstowych. W niektórych zadaniach, między innymi związanych z pozycją wzorca w słowie, przewaga rozwiązania hybrydowego była niewielka lub nie występowała. Szczególnym problemem okazały się zadania wymagające rozpoznawania własności modularnych, takie jak długość słowa modulo określonej wartości czy wartość liczby binarnej modulo  $k$ . Wyniki te pokazują, że samo zastosowanie formalnego algorytmu uczącego nie gwarantuje wysokiej jakości rozwiązania, jeżeli źródłem informacji o języku pozostaje model posiadający ograniczone możliwości rozumowania w danej klasie problemów.

Ważnym wnioskiem z przeprowadzonych badań jest również obserwowana wariancja wyników. Wielokrotne uruchomienie tego samego programu na tym samym modelu mogło prowadzić do różnych rezultatów, mimo braku zmian w kodzie platformy. Wynikało to z niedeterminizmu generowania odpowiedzi przez model językowy oraz wielokrotnego wykorzystywania jego odpowiedzi podczas działania algorytmu  $L^*$ . Zjawisko to ma szczególne znaczenie w przypadku zadań formalnych, ponieważ nawet pojedyncza błędna odpowiedź wyroczni może wpłynąć na kolejne etapy procesu uczenia. Pokazuje to, że w systemach wykorzystujących LLM do zadań wymagających ścisłej poprawności nie można traktować pojedynczej odpowiedzi modelu jako bezwarunkowo wiarygodnego źródła informacji.

Kolejnym elementem badań była próba poprawy wyników poprzez doszkalanie modelu metodą LoRA. W tym celu przygotowano syntetyczny zbiór danych, którego przykłady były generowane na podstawie formalnie zdefiniowanych języków, a następnie niezależnie weryfikowane. Takie rozwiązanie pozwoliło uniknąć ręcznego etykietowania danych

oraz zagwarantować ich zgodność z odpowiednimi strukturami formalnymi. Zastosowano również podział danych według parametrów rodziny języków, dzięki czemu możliwe było sprawdzenie rzeczywistej zdolności modelu do generalizacji.

Eksperyment z doszkalaniem nie doprowadził jednak do pełnego rozwiązania problemu. Model osiągnął 53.6% zgodności na zbalansowanym zbiorze testowym, a dla specjalnie wyłączonego z treningu przypadku modulo 3 uzyskano wynik 50.3%, czyli praktycznie poziom losowego wyboru. Jednocześnie model wykazywał silne przesunięcie w kierunku odpowiedzi przeczących. Wyniki te wskazują, że zastosowane doszkalanie przyniosło ograniczoną poprawę, ale nie pozwoliło modelowi na skuteczne uogólnienie wiedzy na niewidziane wcześniej warianty problemów. W szczególności sugeruje to, że obserwowane ograniczenia nie wynikały wyłącznie z niewystarczającej liczby przykładów treningowych.

Istotnych informacji dostarczyło również porównanie różnych modeli bazowych. Zastosowanie modelu phi4-mini oraz modelu DeepSeek-R1-8B pokazało, że sama zmiana modelu nie rozwiązuje wszystkich problemów platformy. W przypadku języków bezkontekstowych wyniki wszystkich trzech modeli były bardzo zbliżone, co dodatkowo potwierdziło odporność architektury generatywno-weryfikacyjnej. Jednocześnie duża liczba i czas odpowiedzi modelu DeepSeek-R1 powodowały praktyczne problemy w przypadku algorytmu  $L^*$ , który wymaga wielokrotnego odpytywania modelu. Prowadziło to do wyczerpywania limitu czasu zanim algorytm był w stanie zebrać wystarczającą ilość informacji.

Na podstawie przeprowadzonych eksperymentów można zatem sformułować ogólny wniosek dotyczący architektury systemów łączących sztuczną inteligencję z metodami formalnymi. Najbardziej obiecujące okazuje się podejście, w którym model językowy wykorzystywany jest przede wszystkim do interpretacji opisu problemu i wygenerowania kandydata, natomiast jego formalna poprawność jest następnie sprawdzana lub dalsze etapy konstrukcji są realizowane za pomocą deterministycznych algorytmów. Takie rozdzielanie odpowiedzialności pozwala wykorzystać mocne strony modeli językowych, związane z przetwarzaniem języka naturalnego, bez powierzania im całkowitej odpowiedzialności za zachowanie formalnej poprawności konstrukcji.

Wyniki pracy pozwalają również odnieść się bezpośrednio do postawionej hipotezy badawczej. Zakładała ona, że odpowiednio przystosowane duże modele językowe mogą interpretować opisy w języku naturalnym i generować struktury języków formalnych porównywalne pod względem poprawności z konstrukcjami uzyskiwanymi metodami klasycznymi. Hipoteza ta została **częściowo potwierdzona**. W przypadku języków bezkontekstowych zastosowana architektura hybrydowa uzyskała wysoką i stabilną skuteczność, znacząco przewyższającą wyniki surowego modelu językowego. Pokazuje to, że połączenie LLM z klasycznymi metodami formalnymi może być skutecznym sposobem automatycznego tworzenia struktur.

Jednocześnie wyniki dla języków regularnych pokazują, że skuteczność takiego rozwiązania jest zależna od charakteru problemu oraz możliwości wykorzystanego modelu. Szczególnie trudne okazały się zadania wymagające rozumowania modularnego. Ani dalsze modyfikacje algorytmu  $L^*$ , ani niewielkie doszkalanie modelu metodą LoRA nie usunęły w

pełni tych ograniczeń. Oznacza to, że formalna warstwa systemu może skutecznie wykrywać i ograniczać błędy strukturalne, ale nie jest w stanie całkowicie zastąpić kompetencji rozumowania, których brakuje modelowi odpowiedzialnemu za dostarczenie informacji wejściowych.

Przeprowadzone badania pokazują więc, że wykorzystanie sztucznej inteligencji w analizie języków formalnych powinno być traktowane nie jako zastąpienie klasycznych metod, lecz jako ich uzupełnienie. Modele językowe dobrze sprawdzają się jako warstwa komunikacji pomiędzy opisem problemu w języku naturalnym a formalną reprezentacją, natomiast klasyczne algorytmy zapewniają deterministyczne mechanizmy analizy, weryfikacji i przekształcania struktur. Opracowana platforma stanowi przykład takiej architektury i jednocześnie środowisko pozwalające na dalsze badanie tego rodzaju połączeń.

Warto również podkreślić, że jednym z rezultatów pracy nie jest wyłącznie sama implementacja platformy, lecz także identyfikacja ograniczeń poszczególnych sposobów integracji modeli językowych z algorytmami formalnymi. Eksperymenty pokazały, że rozwiązania wykorzystujące wielokrotne zapytania do LLM są szczególnie wrażliwe na czas odpowiedzi, niedeterminizm oraz pojedyncze błędy modelu. Z kolei rozwiązania oparte na jednorazowej generacji i klasycznej weryfikacji wykazały znacznie większą odporność na zmianę modelu bazowego oraz jego szybkości działania. Jest to istotny wniosek projektowy, który może zostać wykorzystany przy dalszym rozwoju platformy.

## 6.1. Ograniczenia i kierunki dalszego rozwoju

Pomimo osiągnięcia założonych celów, przeprowadzone badania posiadają pewne ograniczenia. Eksperymenty zostały wykonane na stosunkowo niewielkim zbiorze 12 języków testowych, obejmującym zarówno proste języki regularne, bardziej wymagające języki regularne, jak i wybrane języki bezkontekstowe. Wyniki należy zatem interpretować jako ocenę zastosowanej architektury w ramach zdefiniowanego zbioru eksperymentalnego, a nie jako pełną charakterystykę możliwości modeli językowych w całej klasie języków formalnych.

Ograniczeniem pozostają również dostępne zasoby obliczeniowe oraz czas wykonywania eksperymentów. Jest to szczególnie widoczne w przypadku modeli generujących rozbudowane odpowiedzi z jawnym procesem rozumowania. W przypadku algorytmu  $L^*$  zwiększenie czasu pojedynczego zapytania przekłada się bezpośrednio na zmniejszenie liczby zapytań możliwych do wykonania w ramach ustalonego budżetu czasowego.

Naturalnym kierunkiem dalszego rozwoju platformy jest rozszerzenie zbioru testowego o większą liczbę rodzin języków oraz bardziej złożone problemy. W szczególności interesujące byłoby zbadanie języków o większej liczbie stanów minimalnych, bardziej złożonych zależnościach modułarnych oraz problemów wymagających jednoczesnego uwzględnienia wielu własności słów.

Kolejnym kierunkiem może być opracowanie bardziej rozbudowanej warstwy weryfikacyjnej, która automatycznie generowałaby kontrprzykłady dla wygenerowanych struktur.

Pozwoliłoby to ograniczyć zależność od ręcznie przygotowanych przypadków testowych i stworzyć bardziej uniwersalny mechanizm wykrywania błędów modeli językowych. Możliwe jest również rozwinięcie podejścia generatywno-weryfikacyjnego poprzez wykorzystanie kilku niezależnych modeli lub różnych sposobów generowania tej samej struktury i porównywanie otrzymanych wyników.

Interesującym kierunkiem dalszych badań pozostaje również wykorzystanie większych lub wyspecjalizowanych modeli językowych oraz bardziej zaawansowanych metod ich adaptacji. Eksperyment z LoRA pokazał, że samo zwiększenie ilości przykładów podobnego typu nie musi prowadzić do poprawy zdolności generalizacji. W przyszłości można zatem zbadać metody treningu ukierunkowane bezpośrednio na rozumowanie symboliczne, generowanie formalnych reprezentacji lub wieloetapowe rozwiązywanie problemów.

Rozbudowy wymagać może także sama warstwa użytkowa platformy. Obecna aplikacja stanowi przede wszystkim środowisko badawcze, jednak jej architektura pozwala na dalsze rozszerzenie funkcjonalności dydaktycznych. Możliwe byłoby między innymi dodanie automatycznego wyjaśniania kolejnych etapów działania algorytmów, generowania zadań dla użytkownika oraz interaktywnego porównywania struktur wygenerowanych przez model z konstrukcjami uzyskanymi metodami klasycznymi.

Podsumowując, opracowana praca pokazuje, że połączenie dużych modeli językowych z klasycznymi metodami teorii języków formalnych jest wykonalne i może prowadzić do praktycznie użytecznych rezultatów. Jednocześnie eksperymenty wskazują, że kluczowe znaczenie ma sposób podziału odpowiedzialności pomiędzy model językowy i algorytm formalny. Model może skutecznie pełnić rolę interfejsu pomiędzy językiem naturalnym a strukturą formalną, jednak zapewnienie wysokiej poprawności wymaga zastosowania niezależnych, deterministycznych mechanizmów analizy i weryfikacji. Właśnie takie połączenie stanowi najważniejszy rezultat przeprowadzonych badań i podstawę do dalszego rozwoju opracowanej platformy.

## Rozdział 7. Instrukcja uruchomienia aplikacji

Opracowana platforma jest aplikacją internetową opartą na technologii ASP.NET Core 8. Do korzystania z funkcji wykorzystujących sztuczną inteligencję wymagane jest dodatkowo lokalne środowisko Ollama oraz pobrany model językowy `phi4-mini`. Niniejszy rozdział przedstawia uproszczoną procedurę przygotowania środowiska i uruchomienia aplikacji.

### 7.1. Wymagania środowiskowe

Do uruchomienia aplikacji potrzebne są:

- system operacyjny umożliwiający uruchomienie platformy .NET 8,
- zainstalowany pakiet .NET 8 SDK,
- zainstalowane środowisko Ollama,
- pobrany model językowy `phi4-mini`,
- przeglądarka internetowa.

Do przygotowania i uruchomienia projektu można wykorzystać środowisko Visual Studio lub terminal systemowy. W przypadku uruchamiania aplikacji z poziomu terminala wymagane jest przejście do katalogu zawierającego plik projektu `FormalStructuresWebApp.csproj`.

### 7.2. Przygotowanie projektu

Po pobraniu kodu źródłowego aplikacji należy rozpakować archiwum z projektem. Następnie w terminalu należy przejść do katalogu:

```
FormalStructuresWebApp/FormalStructuresWebApp
```

W katalogu tym znajduje się plik projektu `FormalStructuresWebApp.csproj`. Przed pierwszym uruchomieniem należy pobrać wymagane zależności projektu za pomocą polecenia:

```
dotnet restore
```

Polecenie pobiera pakiety wymagane przez projekt i przygotowuje środowisko do kompilacji aplikacji.

## 7.3. Instalacja i konfiguracja Ollama

Funkcje sztucznej inteligencji w aplikacji są realizowane z wykorzystaniem lokalnego środowiska Ollama. Ollama udostępnia interfejs API działający lokalnie, z którego aplikacja korzysta podczas wysyłania zapytań do modelu językowego. W konfiguracji aplikacji adres tego interfejsu ustawiono na `http://localhost:11434`, co odpowiada domyślnemu adresowi lokalnego API Ollama.

Ollama należy zainstalować zgodnie z instrukcją dla używanego systemu operacyjnego. Po instalacji poprawność działania można sprawdzić poleceniem:

```
ollama --version
```

Następnie należy pobrać model wykorzystywany przez aplikację:

```
ollama pull phi4-mini
```

Po zakończeniu pobierania dostępne modele można sprawdzić poleceniem:

```
ollama list
```

Na liście powinien znajdować się model `phi4-mini`.

## 7.4. Konfiguracja modelu językowego

Adres Ollama oraz nazwa modelu są przechowywane w pliku `appsettings.json`. Dla wersji aplikacji opisanej w niniejszej pracy konfiguracja powinna zawierać:

```
1 "Ollama": {  
2   "BaseUrl": "http://localhost:11434",  
3   "Model": "phi4-mini"  
4 }
```

Listing 12: Konfiguracja połączenia aplikacji z Ollama

Aplikacja odczytuje te wartości podczas uruchamiania. Klasa `OllamaService` wykorzystuje następnie skonfigurowany adres do wysyłania żądań do endpointu generowania odpowiedzi modelu. Dzięki temu model nie jest uruchamiany bezpośrednio przez aplikację ASP.NET Core, lecz działa jako osobny lokalny proces udostępniający API.

Przed uruchomieniem aplikacji można dodatkowo sprawdzić działanie modelu poleceniem:

```
ollama run phi4-mini
```

Po uruchomieniu modelu można wykonać przykładowe zapytanie w terminalu. Po zakończeniu testu aplikacja może korzystać z tego samego lokalnego środowiska Ollama.

## 7.5. Uruchomienie aplikacji

Po przygotowaniu środowiska oraz pobraniu modelu należy upewnić się, że terminal znajduje się w katalogu zawierającym plik `FormalStructuresWebApp.csproj`. Aplikację uruchamia się poleceniem:

```
dotnet run
```

Podczas uruchamiania aplikacja zostanie skompilowana, a serwer ASP.NET Core rozpocznie nasłuchiwanie na adresach określonych w konfiguracji profili uruchomieniowych. W dostarczonej wersji projektu są to między innymi:

```
https://localhost:7160  
http://localhost:5265
```

Po uruchomieniu serwera jeden z powyższych adresów należy otworzyć w przeglądarce internetowej. Zalecane jest użycie adresu HTTPS:

```
https://localhost:7160
```

Jeżeli przeglądarka wyświetli ostrzeżenie związane z lokalnym certyfikatem deweloperskim HTTPS, certyfikat można zatwierdzić w środowisku deweloperskim lub, w przypadku potrzeby jego ponownego zaufania, wykonać polecenie:

```
dotnet dev-certs https --trust
```

## 7.6. Weryfikacja działania

Po otwarciu aplikacji w przeglądarce należy sprawdzić, czy strona główna została poprawnie wyświetlona. Następnie można przejść do jednego z modułów wykorzystujących model językowy i wykonać przykładowe zadanie generowania lub analizy struktury formalnej. Jeżeli funkcje wykorzystujące sztuczną inteligencję nie działają, w pierwszej kolejności należy sprawdzić, czy środowisko Ollama jest uruchomione oraz czy model `phi4-mini` został pobrany. Pomocne są w tym polecenia:

```
ollama list  
ollama run phi4-mini
```

Jeżeli model nie znajduje się na liście, należy ponownie wykonać:

```
ollama pull phi4-mini
```

W przypadku problemów z uruchomieniem samej aplikacji należy natomiast sprawdzić, czy polecenie `dotnet run` jest wykonywane w katalogu zawierającym plik projektu oraz czy zainstalowana wersja .NET SDK obsługuje platformę .NET 8.

## 7.7. Podstawowa procedura uruchomienia

Po jednorazowym przygotowaniu środowiska kolejne uruchomienia aplikacji wymagają jedynie uruchomienia Ollama oraz aplikacji ASP.NET Core. Minimalna procedura przedstawia się następująco:

1. uruchomić środowisko Ollama,
2. przejść do katalogu zawierającego plik `FormalStructuresWebApp.csproj`,
3. wykonać polecenie `dotnet run`,
4. otworzyć w przeglądarce adres `https://localhost:7160`,
5. korzystać z funkcji platformy, w tym modułów wykorzystujących model `phi4-mini`.

Instalacja zależności projektu za pomocą `dotnet restore` oraz pobranie modelu `phi4-mini` za pomocą `ollama pull` są wymagane przede wszystkim przy pierwszym przygotowaniu środowiska lub po zmianie konfiguracji.

# Bibliografia

## Literatura

- [1] J. E. Hopcroft, R. Motwani, J. D. Ullman, *Introduction to Automata Theory, Languages, and Computation*, 3rd ed., Pearson, 2007.
- [2] M. Sipser, *Introduction to the Theory of Computation*, 3rd ed., Cengage Learning, 2012.
- [3] P. Linz, *An Introduction to Formal Languages and Automata*, 6th ed., Jones & Bartlett Learning, 2016.
- [4] N. Chomsky, “On Certain Formal Properties of Grammars,” *Information and Control*, vol. 2, no. 2, pp. 137–167, 1959.
- [5] A. V. Aho, J. D. Ullman, *The Theory of Parsing, Translation, and Compiling*, Prentice-Hall, 1972.
- [6] K. Thompson, “Programming Techniques: Regular Expression Search Algorithm,” *Communications of the ACM*, vol. 11, no. 6, pp. 419–422, 1968, doi: 10.1145/363347.363387.
- [7] R. McNaughton, H. Yamada, “Regular Expressions and State Graphs for Automata,” *IRE Transactions on Electronic Computers*, vol. EC-9, no. 1, pp. 39–47, 1960.
- [8] D. Angluin, “Learning Regular Sets from Queries and Counterexamples,” *Information and Computation*, vol. 75, no. 2, pp. 87–106, 1987, doi: 10.1016/0890-5401(87)90052-6.
- [9] D. H. Younger, “Recognition and Parsing of Context-Free Languages in Time  $n^3$ ,” *Information and Control*, vol. 10, no. 2, pp. 189–208, 1967.
- [10] T. Kasami, “An Efficient Recognition and Syntax-Analysis Algorithm for Context-Free Languages,” Technical Report, Air Force Cambridge Research Laboratory, 1965.
- [11] T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, *Introduction to Algorithms*, 4th ed., MIT Press, 2022.
- [12] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, I. Polosukhin, “Attention Is All You Need,” in *Advances in Neural Information Processing Systems 30*, 2017.
- [13] T. B. Brown et al., “Language Models Are Few-Shot Learners,” in *Advances in Neural Information Processing Systems 33*, 2020.

- [14] D. Jurafsky, J. H. Martin, *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models*, 3rd ed., online manuscript, 2026.
- [15] W. X. Zhao et al., “A Survey of Large Language Models,” *arXiv preprint arXiv:2303.18223*, 2023.
- [16] L. Ouyang et al., “Training Language Models to Follow Instructions with Human Feedback,” in *Advances in Neural Information Processing Systems 35*, 2022.
- [17] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in *Advances in Neural Information Processing Systems 33*, 2020.
- [18] E. J. Hu et al., “LoRA: Low-Rank Adaptation of Large Language Models,” in *International Conference on Learning Representations*, 2022.
- [19] A. Q. Jiang et al., “Mistral 7B,” *arXiv preprint arXiv:2310.06825*, 2023.
- [20] M. I. Abdin et al., “Phi-4 Technical Report,” Microsoft Research, 2024.
- [21] DeepSeek-AI, D. Guo, D. Yang et al., “DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning,” *arXiv preprint arXiv:2501.12948*, 2025.
- [22] D. Guo et al., “DeepSeek-R1 Incentivizes Reasoning in LLMs through Reinforcement Learning,” *Nature*, vol. 645, pp. 633–638, 2025.
- [23] Microsoft, “ASP.NET Core MVC Overview,” 2026. Available at: <https://learn.microsoft.com/aspnet/core/mvc/overview>
- [24] Ollama, “Ollama API Documentation,” 2026. Available at: <https://docs.ollama.com/api/introduction>
- [25] Hugging Face, “PEFT: Parameter-Efficient Fine-Tuning,” 2026. Available at: <https://huggingface.co/docs/peft>

## Spis listingów

|    |                                                                                         |    |
|----|-----------------------------------------------------------------------------------------|----|
| 1  | Model automatu skończonego wykorzystany w aplikacji . . . . .                           | 28 |
| 2  | Realizacja zapytania o przynależność słowa z wykorzystaniem modelu językowego . . . . . | 30 |
| 3  | Hybrydowy proces generowania automatu DFA na podstawie wyrażenia regularnego . . . . .  | 32 |
| 4  | Kluczowy fragment implementacji algorytmu $L^*$ . . . . .                               | 34 |
| 5  | Sprawdzanie domknięcia i spójności tabeli obserwacji . . . . .                          | 36 |
| 6  | Konstrukcja automatu DFA na podstawie tabeli obserwacji . . . . .                       | 37 |
| 7  | Przygotowanie danych automatu do analizy języka . . . . .                               | 39 |
| 8  | Symulacja automatu ze stosem z wykorzystaniem BFS . . . . .                             | 41 |
| 9  | Konstrukcja automatu ze stosem na podstawie gramatyki bezkontekstowej                   | 43 |
| 10 | Wypełnianie tablicy programowania dynamicznego w algorytmie CYK . . .                   | 45 |
| 11 | Dodawanie stanu w graficznym edytorze automatu . . . . .                                | 47 |
| 12 | Konfiguracja połączenia aplikacji z Ollama . . . . .                                    | 61 |

## Spis tabel

|   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |    |
|---|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 1 | Accuracy klasyfikacji słów testowych po wdrożeniu poprawek (Mistral-7B jako model bazowy). . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | 50 |
| 2 | Porównanie accuracy dla trzech modeli bazowych. * Automaty dla DeepSeek-R1 w językach regularnych są zdegenerowane (1–2 stany) - budżet czasowy L* wyczerpywał się po zaledwie 2–3 z 40 zaplanowanych zapytań ze względu na bardzo długi czas pojedynczej odpowiedzi modelu (rozumowanie krok-po-kroku); wysokie wartości accuracy w tej kolumnie są w dużej mierze artefaktem niezbalansowania klas w teście, nie efektem rzeczywistego uczenia się, i nie powinny być interpretowane jako przewaga modelu. † Przekroczono maksymalny, wydłużony limit czasu żądania (600s). . . . . | 53 |

# Spis wzorów

|                    |    |
|--------------------|----|
| 0. Wzór . . . . .  | 6  |
| 1. Wzór . . . . .  | 6  |
| 2. Wzór . . . . .  | 7  |
| 3. Wzór . . . . .  | 7  |
| 4. Wzór . . . . .  | 7  |
| 5. Wzór . . . . .  | 7  |
| 6. Wzór . . . . .  | 7  |
| 7. Wzór . . . . .  | 7  |
| 8. Wzór . . . . .  | 7  |
| 9. Wzór . . . . .  | 7  |
| 10. Wzór . . . . . | 7  |
| 11. Wzór . . . . . | 7  |
| 12. Wzór . . . . . | 8  |
| 13. Wzór . . . . . | 8  |
| 14. Wzór . . . . . | 8  |
| 15. Wzór . . . . . | 8  |
| 16. Wzór . . . . . | 8  |
| 17. Wzór . . . . . | 9  |
| 18. Wzór . . . . . | 9  |
| 19. Wzór . . . . . | 9  |
| 20. Wzór . . . . . | 9  |
| 21. Wzór . . . . . | 9  |
| 22. Wzór . . . . . | 10 |
| 23. Wzór . . . . . | 10 |
| 24. Wzór . . . . . | 10 |
| 25. Wzór . . . . . | 11 |
| 26. Wzór . . . . . | 11 |
| 27. Wzór . . . . . | 11 |
| 28. Wzór . . . . . | 11 |
| 29. Wzór . . . . . | 11 |
| 30. Wzór . . . . . | 11 |
| 31. Wzór . . . . . | 11 |
| 32. Wzór . . . . . | 11 |
| 33. Wzór . . . . . | 12 |
| 34. Wzór . . . . . | 12 |
| 35. Wzór . . . . . | 12 |
| 36. Wzór . . . . . | 12 |
| 37. Wzór . . . . . | 12 |
| 38. Wzór . . . . . | 13 |

|                    |    |
|--------------------|----|
| 39. Wzór . . . . . | 13 |
| 40. Wzór . . . . . | 13 |
| 41. Wzór . . . . . | 13 |
| 42. Wzór . . . . . | 13 |
| 43. Wzór . . . . . | 13 |
| 44. Wzór . . . . . | 13 |
| 45. Wzór . . . . . | 13 |
| 46. Wzór . . . . . | 13 |
| 47. Wzór . . . . . | 13 |
| 48. Wzór . . . . . | 13 |
| 49. Wzór . . . . . | 13 |
| 50. Wzór . . . . . | 13 |
| 51. Wzór . . . . . | 14 |
| 52. Wzór . . . . . | 14 |
| 53. Wzór . . . . . | 14 |
| 54. Wzór . . . . . | 14 |
| 55. Wzór . . . . . | 14 |
| 56. Wzór . . . . . | 14 |
| 57. Wzór . . . . . | 14 |
| 58. Wzór . . . . . | 14 |
| 59. Wzór . . . . . | 22 |
| 60. Wzór . . . . . | 22 |
| 61. Wzór . . . . . | 22 |
| 62. Wzór . . . . . | 22 |
| 63. Wzór . . . . . | 22 |
| 64. Wzór . . . . . | 23 |
| 65. Wzór . . . . . | 23 |
| 66. Wzór . . . . . | 24 |
| 67. Wzór . . . . . | 24 |
| 68. Wzór . . . . . | 24 |